```
import UIKit
import SDWebImageWebPCoder
import SDWebImage
import FirebaseCrashlytics
import FirebaseCore
import Toast_Swift
import UMCommon
import UMCommon.UMConfigure
import Moya
import os
@main
class AppDelegate: UIResponder, UIApplicationDelegate {
    var window: UIWindow?
    var acceptPrivacy = false
    var retryNetworkMark = false
    var pendingSchemes = [SchemeHandler]()
    var reachabilityTimer: Timer?
    func application(_ application: UIApplication, didFinishLaunchingWithOptions launchOptions: [UIApplication.LaunchOptionsKey: Any]?) -> Bool {
        setupaAppearance()
        window = UIWindow(frame: Utils.currentScreen().bounds)
        window?.makeKeyAndVisible()
        #if DEBUG || ADHOC
        let debugTap = UITapGestureRecognizer(target: self, action: #selector(showDebug(gesture:)))
        debugTap.numberOfTapsRequired = 3
        window?.addGestureRecognizer(debugTap)
        #endif
        showPrivacy {[weak self] in
            UserDefaults.standard.set(true, forKey: "local_did_show_privacy")
            UserDefaults.standard.synchronize()
            self?.acceptPrivacy = true
            self?.appLaunch(application: application, withOptions: launchOptions)
        }
        return true
    }
    func setupaAppearance() {
        UIScrollView.appearance().contentInsetAdjustmentBehavior = .never
        SDImageCodersManager.shared.addCoder(SDImageWebPCoder.shared)
        ToastManager.shared.style.activitySize = CGSize(width: 50, height: 50)
        ToastManager.shared.duration = 60*5
        ToastManager.shared.style.titleFont = .PingFang(ofSize: 14, type: .medium)
        ToastManager.shared.style.messageFont = .PingFang(ofSize: 14, type: .regular)
        ToastManager.shared.style.titleAlignment = .center
        ToastManager.shared.style.messageAlignment = .center
    }
    func loadMain() {
        let login = LoginController.tryGetLogin()
        let rootNav = NavController(rootViewController: HomeController(querys: nil))
        if let login = login {
            rootNav.pushViewController(login, animated: false)
```

```
            }
            window?.rootViewController = rootNav
            MainDataManager.shared.requestInit()// 直接发起 v1/init 请求
            pendingSchemes.forEach({$0.execute()})
            pendingSchemes.removeAll()
            //AppVersionHelper.checkUpgrade()
        }
        @objc func showDebug(gesture: UITapGestureRecognizer) {
            SchemeHandler(type: .debug, querys: ["nologin":"1"]).execute()
        }
}
//MARK: - Launch
extension AppDelegate {
    func appLaunch(application: UIApplication, withOptions options:[UIApplication.LaunchOptionsKey: Any]?) {
            let launch = LaunchController(querys: nil)
            launch.delegate = self
            window?.rootViewController = launch
            startLaunchApp()
        }
        func startLaunchApp() {
            if setupReachability() {
                networkReady()
            } else {
                reachabilityTimer = Timer.scheduledTimer(withTimeInterval: 10, repeats: false, block: {[weak self] t in
                    t.invalidate()
                    self?.reachabilityTimer = nil
                    self?.startLaunchLogic()
                })
            }
            PushManager.shared.requestAuthorization() // 请求通知权限
        }
        func didLaunchApp() {
            UserAgent.shared.loadWebUserAgent()
            DeviceManager.shared.updateDevice()
            reloadHighLevelAPI()
            registerThirdPart()
            startLaunchLogic()
            if let deviceId = DeviceManager.shared.deviceId, !deviceId.isEmpty {
                PasteboardManager.shared.state.insert(.devicePrepared)
                Crashlytics.crashlytics().setCustomValue(deviceId, forKey: "device_id")
            }
            TrackManager.shared.openApp() // 冷启动的 openApp
        }
        func reloadHighLevelAPI() {
            DeviceManager.shared.deviceUpdated()
            if User.shared.didLogin {
                NetClient.requestJson(target: .renew, success: nil, failure: { error in
                    if let netError = error as? NetworkError {
                        switch netError {
```

```
                        case .invalidStatus(_):
                            User.shared.logout()
                        case .limitUse(_,_):
                            break
                        }
                    }
                }, completion: nil)
            }
            //AAAManager.shared.postAttribution()
        }
        func registerThirdPart() {
            FirebaseApp.configure()
            JDManager.shared.initSDK()
            BaichuanManager.shared.setupBaichuan()
//              ShareManager.shared.registerApp()
//              ChatIFLYManager.shared.initSDK()
            UMConfigure.initWithAppkey(UMAppKey, channel: "App Store")
//              OpeninstallManager.shared.registerOpenInstall()
        }
        func startLaunchLogic() {
            var launch: LaunchController
            if let root = window?.rootViewController as? LaunchController {
                launch = root
            } else {
                launch = LaunchController(querys: nil)
                launch.delegate = self
                window?.rootViewController = launch
            }
            launch.start()
        }
}
// MARK: - Launch
extension AppDelegate: LaunchControllerDelegate {
        func launchDidFinish(launch: LaunchController) {
            loadMain()
            /*
            let didShowWelcome = UserDefaults.standard.bool(forKey:
"local_did_show_welcome")
            if didShowWelcome {// step1: 展示过欢迎直接进入
                loadMain()
                return
            }
            // step2: 没有展示过 welcome 根据服务端字段控制是否展示
            UserDefaults.standard.set(true, forKey: "local_did_show_welcome")
            UserDefaults.standard.synchronize()
            let welcome = WelcomeController(querys: nil) // 没有看过欢迎引导，展示欢迎引导
            welcome.delegate = self
            window?.rootViewController = welcome
            */
        }
}
```

```
/*
extension AppDelegate: WelcomeControllerDelegate {
    func welcomeFinished(welcome: WelcomeController?) -> Void {
        loadMain()
    }
}
*/
// MARK: - Privacy
extension AppDelegate {
    func didShowPrivacy() -> Bool {
        let didShow = UserDefaults.standard.bool(forKey: "local_did_show_privacy")
        return didShow
    }
    func showPrivacy(completion: (()->Void)?) {
        guard !didShowPrivacy() else {
            completion?()
            return
        }
        let vc = PrivacyController(complete: completion)
        vc.finishCompletion = completion
        let nav = NavController(rootViewController: vc)
        window?.rootViewController = nav
    }
}
//MARK: - Reachability
extension AppDelegate {
    func setupReachability() -> Bool {
        NotificationCenter.default.addObserver(self, selector:
#selector(networkStatusChange(notify:)), name: .networkFlagsChanged, object: nil)
        do {
            try ReachabilityNetwork.reachability = Reachability(hostname: "www.apple.com")
            let result = ReachabilityNetwork.reachability.isConnectedToNetwork
            retryNetworkMark = !result
            return result
        }
        catch {
            retryNetworkMark = true
            return false
        }
    }
    @objc func networkStatusChange(notify: Notification) {
        guard retryNetworkMark && ReachabilityNetwork.reachability.isConnectedToNetwork
else {
            return
        }
        if let t = reachabilityTimer {
            t.invalidate()
            reachabilityTimer = nil
        }
        retryNetworkMark = false//  只处理一次
        networkReady()
```

```
    }
    func networkReady() {
        requestStartAPI {[weak self] in
            self?.didLaunchApp()
        }
    }
    /// 请求 start 接口，这个是初始化接口，也是最早的接口
    /// - Parameter completion: 完成回调
    func requestStartAPI(completion: (()->Void)?) {
        let provider = MoyaProvider<NetAPI>(requestClosure: { (endpoint: Endpoint, done:
@escaping MoyaProvider<NetAPI>.RequestResultClosure) in
            guard var request = try? endpoint.urlRequest() else {
                done(.failure(MoyaError.requestMapping(endpoint.url)))
                return
            }
            request.timeoutInterval = 10       //设置请求超时时间
            done(.success(request))
        })
#if DEBUG || ADHOC
        os_log(.default,                    log:.normal,"API                    start
request : %{public}.5f",Date().timeIntervalSince1970)
#endif
        provider.request(.start, callbackQueue: DispatchQueue.main) { result in
#if DEBUG || ADHOC
            os_log(.default,                    log:.normal,"API                    start
response: %{public}.5f",Date().timeIntervalSince1970)
#endif
            switch result {
            case .success(let resp):
                do {
                    let startResp = try resp.map(StartResponse.self)
                    if let data = startResp.data {
                        MainDataManager.shared.startData = data
                        if let cj = data.caiJi, cj == 1 { // 可以生成对象
                            DeviceManager.shared.hwInfoObject = HWInfo()
                        }
                    }
                } catch MoyaError.objectMapping(let err, resp) {
                    print("map start resp failed:\(err.localizedDescription)")
                } catch {
                    print("map start resp failed:\(error.localizedDescription)")
                }
                completion?() // 进入下一步
            case .failure(let moyaError):
                print("error:\(moyaError.localizedDescription)")
                completion?() // 进入下一步
            }
        }
    }
}
// MARK: - App Life Cycle Event
```

```
extension AppDelegate {
    func applicationDidBecomeActive(_ application: UIApplication) {
        guard acceptPrivacy else {return}
//          AppStateManager.shared.handleDidBecomeActive()
        if DeviceManager.shared.startByInit && (DeviceManager.shared.idfa?.isEmpty ?? true)
{
            DeviceManager.shared.updateIDFA(nil)
        }
        PasteboardManager.shared.state.insert(.appActive)
    }
    func applicationDidEnterBackground(_ application: UIApplication) {
        guard acceptPrivacy else {return}
        TrackManager.shared.closeApp()
        Utils.updateLastVersion()
        PasteboardManager.shared.state.remove(.appActive)
        PasteboardManager.shared.appEnterBackground()
    }
    func applicationWillResignActive(_ application: UIApplication) {
        guard acceptPrivacy else {return}
    }
    func applicationWillEnterForeground(_ application: UIApplication) {
        DeviceManager.shared.startByInit = false
        guard acceptPrivacy else {return}
        TrackManager.shared.openApp()
        reloadHighLevelAPI()
    }
    func applicationWillTerminate(_ application: UIApplication) {
        guard acceptPrivacy else {return}
        Utils.updateLastVersion()
        PasteboardManager.shared.state.remove(.appActive)
        PasteboardManager.shared.appWillTerminate()
    }
    func applicationDidReceiveMemoryWarning(_ application: UIApplication) {
    }
    func application(_ app: UIApplication, open url: URL, options:
[UIApplication.OpenURLOptionsKey : Any] = [:]) -> Bool {
        guard acceptPrivacy else {return false}
        if let scheme = url.scheme, scheme == KeplerScheme { // JD Kepler
            let result = JDManager.shared.handleCallbackUrl(url: url)
            if result {
                return true
            }
        }
        if let scheme = url.scheme, scheme == BaichuanScheme { //  处理百川的 scheme
            let result = BaichuanManager.shared.handleBaichuanSchemeOpen(app, open: url,
options: options)
            if result {
                return true
            }
        }
        if let scheme = url.scheme, scheme == UMengScheme    {
```

```
            return MobClick.handle(url)
        }
        /*if let scheme = url.scheme, scheme == WechatAppId {
            return ShareManager.shared.handleUrl(url)
        }
        OpeninstallManager.shared.excuteWakeUpLink = true
        if OpeninstallManager.handleLinkURL(url: url) {// 处理 openinstall
            return true
        }*/
        guard let scheme = SchemeHandler(url) else { return false }
        guard PageManager.getRootNav() != nil else {
            pendingSchemes.append(scheme)
            return false
        }
        scheme.execute()
        return true
    }
    func application(_ application: UIApplication, continue userActivity: NSUserActivity, restorationHandler: @escaping ([UIUserActivityRestoring]?) -> Void) -> Bool {
        guard acceptPrivacy else {return false}
        /*if ShareManager.shared.handleUniversalLink(userActivity: userActivity) {
            return true
        }*/
        /*
        OpeninstallManager.shared.excuteWakeUpLink = true
        if OpeninstallManager.continueUserActivity(userActivity: userActivity) {// 处理 openinstall
            return true
        }
        // 处理自己的 universal link
        guard let webpageURL = userActivity.webpageURL else { return false }
        let webpageURLStr = webpageURL.absoluteString.lowercased()
        guard webpageURLStr.hasPrefix("https://"+appHost) || webpageURLStr.hasPrefix("http://"+appHost) else {return false} // 不是自己的 ulink
        guard webpageURL.path == "/ulink/scheme" else {return true} //是自己的 universal link，但是 path 不是用来执行 scheme 的
        guard let appLinkStr = webpageURL.queryValue(for: "go"), let link = URL(string: appLinkStr), let scheme = CustomSchemeHandler(link) else {return true} // 参数出错
        scheme.execute()
         */
        return true
    }
}
import UIKit
import SnapKit
class BaseController: UIViewController {
    public var nonav = false
    public var noback = false
    public var nologin = false
    public var pageName: String?
    public var statusStyle: UIStatusBarStyle = .default
```

```
    let uuid: String = UUID().uuidString
    let lastUUID: String?
    open var navView = BaseNavView()
    public let navBgView = UIView()
    var pushPopTransition: PushPopTransition?
    private(set) var viewDidShow = false
    var weakChilds = [WeakWrapped<UIViewController>]()
    override var preferredStatusBarStyle: UIStatusBarStyle {
        return statusStyle
    }
    var showBack: Bool {
        if let first = navigationController?.viewControllers.first, first != self {
            return true
        }
        return false
    }
    override var prefersHomeIndicatorAutoHidden: Bool {
        return true
    }
    public init(querys: [String: String]?) {
        if let delegate = UIApplication.shared.delegate as? AppDelegate, let root = delegate.window?.rootViewController, let top = PageManager.getTopController(with: root) as? BaseController {
            lastUUID = top.uuid
        } else {
            lastUUID = nil
        }
        super.init(nibName: nil, bundle: nil)
        hidesBottomBarWhenPushed = true
        modalPresentationStyle = .fullScreen
        handle(paramaters: querys)
    }
    required init?(coder: NSCoder) {
        fatalError("init(coder:) has not been implemented")
    }
    override func viewDidLoad() {
        super.viewDidLoad()
        view.backgroundColor = "#F1F5FF".uicolor()
        setupNav()
    }
    override func viewWillAppear(_ animated: Bool) {
        super.viewWillAppear(animated)
        viewDidShow = false
    }
    override func viewDidAppear(_ animated: Bool) {
        super.viewDidAppear(animated)
        viewDidShow = true
        Track.event("p_show", attributes: ["page_name":pageName ?? String(describing: self.self),"uuid":uuid,"last_uuid":lastUUID ?? ""])
    }
    override func viewDidDisappear(_ animated: Bool) {
```

```
        super.viewDidDisappear(animated)
        Track.event("p_hide", attributes: ["page_name":pageName ?? String(describing: self.self),"uuid":uuid,"last_uuid":lastUUID ?? ""])
    }
    func handle(paramaters: [String: String]?) {
        guard let paramaters = paramaters else { return }
        if let nologinStr = paramaters["nologin"], nologinStr == "1" {
            nologin = true
        }
        if let nonavStr = paramaters["nonav"], nonavStr == "1" {
            nonav = true
        }
        if let noBackStr = paramaters["noback"], noBackStr == "1" {
            noback = true
        }
        if let pnStr = paramaters["pn"] {
            pageName = pnStr
        }
    }
    func setupNav() {
        navBgView.backgroundColor = "#F1F5FF".uicolor()
        view.addSubview(navBgView)
        navBgView.addSubview(navView)
        navView.snp.makeConstraints { make in
            make.leading.trailing.equalToSuperview()
            make.height.equalTo(44)
            make.top.equalTo(view.safeAreaLayoutGuide.snp.top)
        }
        navView.backBtn.addTarget(self, action: #selector(clickBackBtn(sender:)), for: .touchUpInside)
        navView.hideBack = !(showBack && !noback)
        navBgView.snp.makeConstraints { make in
            make.top.leading.trailing.equalToSuperview()
            make.bottom.equalTo(navView)
        }
        if nonav {
            navBgView.isHidden = true
        } else {
            navBgView.isHidden = false
        }
    }
    @objc open func clickBackBtn(sender: UIButton) {
        pop()
    }
}
class BaseNavView: UIView {
    let titleLabel = UILabel()
    let backBtn = NavIconButton(inset: UIEdgeInsets(top: 5, left: 5, bottom: 5, right: 5))
    var hideBack = false {
        didSet {
            backBtn.isHidden = hideBack
```

```
            }
        }
        var title: String? {
            didSet {
                titleLabel.text = title
            }
        }
        override init(frame: CGRect) {
            super.init(frame: frame)
            addSubview(titleLabel)
            addSubview(backBtn)
            backBtn.icon.image = UIImage(named: "nav_back_black")
            titleLabel.snp.makeConstraints { make in
                make.center.equalToSuperview()
                make.leading.equalTo(backBtn.snp.trailing).offset(2)
            }
            titleLabel.font = .PingFang(ofSize: 16, type: .semibold)
            titleLabel.textColor = "#333333".uicolor()
            titleLabel.numberOfLines = 1
            titleLabel.lineBreakMode = .byTruncatingTail
            titleLabel.textAlignment = .center
            backBtn.snp.makeConstraints { make in
                make.centerY.equalToSuperview()
                make.leading.equalTo(10)
            }
        }
        required init?(coder: NSCoder) {
            fatalError("init(coder:) has not been implemented")
        }
}
class NavBaseButton: UIControl {
        let content = UIView()
        let btnInset: UIEdgeInsets
        init(inset: UIEdgeInsets) {
            btnInset = inset
            super.init(frame: .zero)
            addSubview(content)
            content.isUserInteractionEnabled = false
            content.snp.makeConstraints { make in
                make.edges.equalToSuperview().inset(btnInset)
            }
        }
        required init?(coder: NSCoder) {
            fatalError("init(coder:) has not been implemented")
        }
}
class NavIconButton: NavBaseButton {
        let icon = UIImageView()
        override init(inset: UIEdgeInsets) {
            super.init(inset: inset)
            content.addSubview(icon)
```

```
            icon.snp.makeConstraints { make in
                make.size.equalTo(CGSize(width: 30, height: 30))
                make.edges.equalToSuperview()
            }
        }
        required init?(coder: NSCoder) {
            fatalError("init(coder:) has not been implemented")
        }
}
import UIKit
class NavController: UINavigationController {
        override init(rootViewController: UIViewController) {
            super.init(rootViewController: rootViewController)
            modalPresentationStyle = .fullScreen
            interactivePopGestureRecognizer?.delegate = self
        }
        override init(nibName nibNameOrNil: String?, bundle nibBundleOrNil: Bundle?) {
            super.init(nibName: nibNameOrNil, bundle: nibBundleOrNil)
            modalPresentationStyle = .fullScreen
            interactivePopGestureRecognizer?.delegate = self
        }
        required init?(coder aDecoder: NSCoder) {
            fatalError("init(coder:) has not been implemented")
        }
        override func viewDidLoad() {
            super.viewDidLoad()
            view.backgroundColor = .white
            setNavigationBarHidden(true, animated: false)
        }
        override var preferredStatusBarStyle: UIStatusBarStyle {
            return topViewController?.preferredStatusBarStyle ?? .default
        }
}
extension NavController: UIGestureRecognizerDelegate { //  处理侧滑返回
        func gestureRecognizerShouldBegin(_ gestureRecognizer: UIGestureRecognizer) -> Bool {
            guard gestureRecognizer == interactivePopGestureRecognizer else {
                return true
            }
            if let first = viewControllers.first, first == visibleViewController, viewControllers.count
< 2 {
                return false
            }
            return true
        }
}
import UIKit
enum AnimatorTransitionType {
        case push
        case pop
        case present
        case dismiss
```

```
}
class PushPopTransition: AnimatorBaseTransition {
    var operation: UINavigationController.Operation = .none
    override func animateTransition(using transitionContext:
UIViewControllerContextTransitioning) {
        super.animateTransition(using: transitionContext)
        switch operation {
        case .push:
            animationPush()
        case .pop:
            animationPop()
        default:
            break
        }
    }
    open func animationPush() {
    }
    open func animationPop() {
    }
    override func animationEnded(_ transitionCompleted: Bool) {
        super.animationEnded(transitionCompleted)
    }
    override func transitionDuration(using transitionContext:
UIViewControllerContextTransitioning?) -> TimeInterval {
        return 0.5
    }
}
class AnimatorBaseTransition: NSObject, UIViewControllerAnimatedTransitioning {
    weak var transitionContext: UIViewControllerContextTransitioning?
    var containerView: UIView!
    var from: UIViewController?
    var to: UIViewController?
    var fromView: UIView?
    var toView: UIView?
    open func transitionDuration(using transitionContext:
UIViewControllerContextTransitioning?) -> TimeInterval {
        return 0.5
    }
    open func animateTransition(using transitionContext: UIViewControllerContextTransitioning)
{
        self.transitionContext = transitionContext
        containerView = transitionContext.containerView
        from = transitionContext.viewController(forKey: .from)
        to = transitionContext.viewController(forKey: .to)
        fromView = transitionContext.view(forKey: .from)
        toView = transitionContext.view(forKey: .to)
    }
    func animationEnded(_ transitionCompleted: Bool) {
    }
}
import UIKit
```

```
class TabController: UITabBarController {
    override func viewDidLoad() {
        super.viewDidLoad()
        // Do any additional setup after loading the view.
    }
    /*
    // MARK: - Navigation
    // In a storyboard-based application, you will often want to do a little preparation before navigation
    override func prepare(for segue: UIStoryboardSegue, sender: Any?) {
        // Get the new view controller using segue.destination.
        // Pass the selected object to the new view controller.
    }
    */
}
import UIKit
import Moya
protocol InitDataDelegate: AnyObject {
    func initData(data: MainDataManager, receiveData: InitDataResponse)
    func initData(data: MainDataManager, failedWithError error: Error)
}
class MainDataManager {
    static let shared = MainDataManager()
    private var initToken: Cancellable?
    private var configToken: Cancellable?
    weak var delegate: InitDataDelegate?
    var startData: StartData?
    var initData: InitResponseData?
    var configData: ConfigData?
    var isGuest: Bool {
        return configData?.guest ?? true
    }
    func requestInit() {
        cancelInit()
        initToken = NetClient.request(target: .v1Init, success: {[weak self] (data: InitDataResponse, resp) in
            guard let strongSelf = self else { return }
            strongSelf.initData = data.data
            strongSelf.delegate?.initData(data: strongSelf, receiveData: data)
            var info: [String: Any] = ["result":true]
            if let d = data.data {
                info["data"] = d
            }
            NotificationCenter.default.post(name: .initFinish, object: nil, userInfo: info)
        }, failure: {[weak self] error in
            guard let strongSelf = self else { return }
            strongSelf.delegate?.initData(data: strongSelf, failedWithError: error)
            NotificationCenter.default.post(name: .initFinish, object: nil, userInfo: ["result":false])
        })
    }
```

```
    func cancelInit() {
        guard let req = initToken else { return }
        req.cancel()
        initToken = nil
    }
    func requestConfig() {
        cancelConfig()
        configToken = NetClient.request(target: .appConfig, success: { [weak self] (data: ConfigResponse, resp) in
            guard let strongSelf = self else { return }
            strongSelf.configData = data.data
            var info: [String: Any] = ["result":true]
            if let d = data.data {
                info["data"] = d
            }
            NotificationCenter.default.post(name: .configFinish, object: nil, userInfo: info)
        }, failure: { error in
            NotificationCenter.default.post(name: .configFinish, object: nil, userInfo: ["result":false])
        })
    }
    func cancelConfig() {
        guard let req = configToken else { return }
        req.cancel()
        configToken = nil
    }
}
struct StartResponse: Codable {
    let data: StartData?
    let info: String?
    let status: Int?
}
struct StartData: Codable {
    let caiJi: Int?
}
struct InitDataResponse: Codable {
    let data: InitResponseData?
    let info: String?
    let status: Int?
}
enum HomeActionButtonConfig: Int, Codable {
    case onlyQuanWangBiJia = 1
    case bijiaAndKanJia = 2
}
struct InitResponseData: Codable {
    let pages: InitDataPages?
    let inputHint: String?
    let bottomBanner: InitBottomBanner?
    let actionButtonConfig: HomeActionButtonConfig?
    let notification: NotificationRedPointData?
}
```

```
struct InitDataPages: Codable {
    let center: String?
}
struct InitBottomBanner: Codable {
    let items: [InitBottomItem]?
}
struct InitBottomItem: Codable {
    let image: ImageData?
}
struct ImageData: Codable {
    struct ImageSize: Codable {
        let width: Double?
        let height: Double?
    }
    let url: String?
    let size: ImageSize?
}
struct ConfigResponse: Codable {
    let data: ConfigData?
    let info: String?
    let status: Int?
}
struct ConfigData: Codable {
    let guest: Bool?
    let showTab: Bool?
    let is_adv: Bool?
    let kouLingLength: Int?
}
struct NotificationRedPointData: Codable {
    let center: Int?
}
struct NotificationRedPointResponse: Codable {
    let data: NotificationRedPointData?
    let info: String?
    let status: Int?
}
import UIKit
import os
import Toast_Swift
import SnapKit
import TBGrowingTextView
import SDWebImage
class HomeController: BaseController {
    let homeView = HomeView()
    var lastTextEmpty: Bool = true
    var pasteData: PasteData?
    override init(querys: [String : String]?) {
        super.init(querys: querys)
        MainDataManager.shared.delegate = self
        NotificationCenter.default.addObserver(self, selector:
#selector(addRedPoint(notification:)), name: .addRedPoint, object: nil)
```

```
        NotificationCenter.default.addObserver(self, selector:
#selector(clearRedPoint(notification:)), name: .clearRedPoint, object: nil)
    }
    required init?(coder: NSCoder) {
        fatalError("init(coder:) has not been implemented")
    }
    deinit {
        NotificationCenter.default.removeObserver(self)
    }
    override func loadView() {
        view = homeView
        homeView.delegate = self
    }
    override func viewDidLoad() {
        super.viewDidLoad()
        title = "Home"
        let needHideGuide = UserDefaults.standard.bool(forKey: "local_need_hide_guide")
        homeView.guideView.delegate = self
        homeView.guideView.isHidden = needHideGuide
        if needHideGuide {
            self.homeView.bottomMaxCons?.update(offset: 100)
        }
    }
    override func viewWillAppear(_ animated: Bool) {
        super.viewWillAppear(animated)
    }
    override func viewDidAppear(_ animated: Bool) {
        super.viewDidAppear(animated)
        PasteboardManager.shared.state.insert(.homeDidShow)
    }
    override func viewWillDisappear(_ animated: Bool) {
        super.viewWillDisappear(animated)
        PasteboardManager.shared.state.remove(.homeDidShow)
    }
    override func handle(paramaters: [String : String]?) {
        super.handle(paramaters: paramaters)
        nonav = true
        nologin = true
        pageName = "main"
    }
    func reset() {
        MainDataManager.shared.requestInit()
    }
    func requestDispatch(type: DispatchType) {
        guard let txt = homeView.inputText, !txt.isEmpty else {
            view.hideAllToasts(includeActivity: true)
            view.makeToast("请粘贴商品链接或输入商品名称", duration: 1.5,
position: .center)
            return
        }
        view.hideAllToasts(includeActivity: true)
```

```
            showLoading()
            homeView.isUserInteractionEnabled = false
            let hasPasteData = pasteData != nil
            NetClient.request(target:  .v1Dispatch(type,txt,pasteData?.jsonString())) { [weak self]
(dispatchResp: DispatchResponse, resp) in
                self?.view.hideAllToasts(includeActivity: true)
                self?.homeView.isUserInteractionEnabled = true
                let valid = dispatchResp.data?.valid ?? 0 == 1
                if valid, let data = dispatchResp.data, let link = data.link, !link.isEmpty, let url =
URL(string: link), let strongSelf = self {
                    SchemeHandler(url)?.execute(SchemeContext(controller: strongSelf))
                    if hasPasteData {
                        strongSelf.hideHomeGuide()
                    }
                }
                if !valid, let msg = dispatchResp.data?.toast, !msg.isEmpty {
                    self?.view.makeToast(msg, duration: 1.5, position: .center)
                }
            } failure: {[weak self] error in
                self?.view.hideAllToasts(includeActivity: true)
                self?.homeView.isUserInteractionEnabled = true
                if let strongSelf = self, let netError = error as? NetworkError {
                    switch netError {
                    case .invalidStatus(let info):
                        if let infoMsg = info["info"] as? String {
                            strongSelf.view.makeToast(infoMsg, duration: 1.5, position: .center)
                        }
                        break
                    case .limitUse(_, _):
                        break
                    }
                }
            }
        }
        func showLoading() {
            let imageView = SDAnimatedImageView(image: SDAnimatedImage(named:
"loading_v1"))
            imageView.frame = CGRect(origin: .zero, size: CGSize(width: 90, height: 90))
            view.showToast(imageView, duration: 5 * 60,position: .center)
        }
        func requestPaste(contnet: String) {
            view.endEditing(true)
           view.hideAllToasts(includeActivity: true)
            showLoading()
            homeView.isUserInteractionEnabled = false
            NetClient.request(target: .paste(contnet)) { [weak self] (pasteResp: PasteResponse, resp)
in
                self?.view.hideAllToasts(includeActivity: true)
                self?.homeView.isUserInteractionEnabled = true
                self?.pasteData = pasteResp.data
                self?.homeView.updatePasteData(data: pasteResp.data)
```

```
                let valid = pasteResp.data?.valid ?? 0 == 1
                if !valid, let msg = pasteResp.data?.toast, !msg.isEmpty {
                    self?.view.makeToast(msg, duration: 1.5, position: .center)
                }
            } failure: {[weak self] error in
                self?.view.hideAllToasts(includeActivity: true)
                self?.homeView.isUserInteractionEnabled = true
                if let strongSelf = self, let netError = error as? NetworkError {
                    switch netError {
                    case .invalidStatus(let info):
                        if let infoMsg = info["info"] as? String {
                            strongSelf.view.makeToast(infoMsg, duration: 1.5, position: .center)
                        }
                        break
                    case .limitUse(_, _):
                        break
                    }
                }
            }
        }
        func canSendMsg() -> Bool {
            return true
        }
        // MARK: - Notification
        @objc func addRedPoint(notification:Notification) {
            homeView.nav.mineIcon.isHighlighted = true
        }
        @objc func clearRedPoint(notification:Notification) {
            homeView.nav.mineIcon.isHighlighted = false
        }
}
extension HomeController: HomeViewDelegate {
        func homeView(homeView: HomeView, textDidChanged input: GrowingTextView) {
            if lastTextEmpty, input.hasText, let text = input.text, !text.isEmpty, text.count >=
MainDataManager.shared.configData?.kouLingLength ?? 13 {// 触发 paste 接口
                requestPaste(contnet: text) // request paste api
            }
            lastTextEmpty = !input.hasText
            if lastTextEmpty { // 清空了输入框，则干掉当前保存的 pasteData。避免 dispatch 接
口的 pasteInfo 错乱
                pasteData = nil
                homeView.resetInput()
            }
        }
        func homeView(homeView:HomeView, didClickMineButton button: UIButton) {
            guard let center = MainDataManager.shared.initData?.pages?.center, !center.isEmpty, let
url = URL(string: center) else { return }
            if let scheme = url.scheme, scheme.hasPrefix("http") {
                SchemeHandler(type:                .openWeb,                querys:
["url":center,"nonav":"1"]).execute(SchemeContext(controller: self))
            } else {
```

```
                SchemeHandler(url)?.execute(SchemeContext(controller: self))
            }
        }
        func homeView(homeView: HomeView, clickBiJia button: HomeActionButton) {
            requestDispatch(type: .quanWangBiJia)
        }
        func homeView(homeView: HomeView, clickKanJia button: HomeActionButton) {
            Track.btnClick(btnName: "main_start_kanjia")
            guard !KanjiaMiniModeManager.shared.floatExist() else {
                view.makeToast("您有正在砍价的商品\n 砍价结束后才能开启新商品砍价",
duration: 1.5 ,position: .center)
                return
            }
            requestDispatch(type: .AIKanJia)
        }
        func homeView(homeView: HomeView, shouldTextViewReturn growingTextView:
GrowingTextView) -> Bool {
//            guard growingTextView.hasText, canSendMsg() else {return false}
            return false
        }
        func homeView(homeView: HomeView, shouldTextViewEditing growingTextView:
GrowingTextView) -> Bool {
            Track.btnClick(btnName: "main_input")
            return true
        }
}
extension HomeController: InitDataDelegate {
        func initData(data: MainDataManager, receiveData: InitDataResponse) {
            homeView.updateInitData(data: receiveData.data)
        }
        func initData(data: MainDataManager, failedWithError error: Error) {
            if let netError = error as? NetworkError {
                switch netError {
                case .invalidStatus(let info):
                    if let infoMsg = info["info"] as? String {
                        view.hideAllToasts(includeActivity: true)
                        view.makeToast(infoMsg, duration: 1.5, position: .center)
                    }
                    break
                case .limitUse(_, _):
                    break
                }
            }
        }
}
extension HomeController: HomeGuideViewDelegate {
        func homeGuide(guide: HomeGuideView, didClickClose: UIButton) {
            hideHomeGuide()
        }
        func homeGuide(guide: HomeGuideView, didClickTaoBao: UIButton) {
            jumpToOther(scheme: "tbopen://", h5: "https://main.m.taobao.com/")
```

```
    }
    func homeGuide(guide: HomeGuideView, didClickJD: UIButton) {
        jumpToOther(scheme: "openapp.jdmobile://", h5: "https://m.jd.com/")
    }
    func homeGuide(guide: HomeGuideView, didClickPdd: UIButton) {
        jumpToOther(scheme: "pddopen://", h5: "https://m.pinduoduo.com/home/")
    }
    func jumpToOther(scheme: String, h5: String) {
        guard let schemeUrl = URL(string: scheme), UIApplication.shared.canOpenURL(schemeUrl) else {
            SchemeHandler(type: .openWeb, querys: ["url":h5]).execute(SchemeContext(controller: self))
            return
        }
        UIApplication.shared.open(schemeUrl) { success in
            #if DEBUG
            print("jump \(schemeUrl.scheme!) success: \(success ? "true" : "false")")
            #endif
        }
    }
    func hideHomeGuide() {
        guard !homeView.guideView.isHidden else {return}
        UserDefaults.standard.set(true, forKey: "local_need_hide_guide")
        UserDefaults.standard.synchronize()
        UIViewPropertyAnimator.runningPropertyAnimator(withDuration: 0.2, delay: 0) {
            self.homeView.guideView.isHidden = true
            self.homeView.bottomMaxCons?.update(offset: 100)
        }
    }
}
import UIKit
import SnapKit
import SDWebImage
import TBGrowingTextView
protocol HomeViewDelegate: AnyObject {
    func homeView(homeView:HomeView, didClickMineButton button: UIButton)
    func homeView(homeView: HomeView, textDidChanged input: GrowingTextView)
    func homeView(homeView: HomeView, clickBiJia button: HomeActionButton)
    func homeView(homeView: HomeView, clickKanJia button: HomeActionButton)
    func homeView(homeView: HomeView, shouldTextViewReturn growingTextView: GrowingTextView) -> Bool
    func homeView(homeView: HomeView, shouldTextViewEditing growingTextView: GrowingTextView) -> Bool
}
// 导航栏
class HomeNavView: UIView {
    let navView = UIView()
    let mineIcon = UIImageView(image: UIImage(named: "home_mine"), highlightedImage: UIImage(named: "home_mine_red"))
    let mineBtn = UIButton()
    override init(frame: CGRect) {
```

```
            super.init(frame: frame)
            setupUI()
        }
        required init?(coder: NSCoder) {
            fatalError("init(coder:) has not been implemented")
        }
        func setupUI() {
            backgroundColor = .clear
            addSubview(navView)
            navView.snp.makeConstraints { make in
                make.leading.trailing.bottom.equalToSuperview()
                make.height.equalTo(44)
                make.top.equalTo(safeAreaLayoutGuide.snp.top)
            }
            navView.addSubview(mineIcon)
            mineIcon.snp.makeConstraints { make in
                make.size.equalTo(CGSize(width: 30, height: 30))
                make.right.equalTo(-10)
                make.centerY.equalToSuperview()
            }
            navView.addSubview(mineBtn)
            mineBtn.snp.makeConstraints { make in
                make.edges.equalTo(mineIcon)
            }
#if DEBUG
            if #available(iOS 15.0, *) {
                var configuration = UIButton.Configuration.filled()
                configuration.baseBackgroundColor = .clear
                configuration.attributedTitle = AttributedString()
                configuration.image = UIImage(systemName: "wrench.and.screwdriver.fill")
                let debugBtn = UIButton(configuration: configuration, primaryAction:
UIAction(handler: { _ in
                    SchemeHandler(type: .debug,querys: [String:String]()).execute()
                }))
                navView.addSubview(debugBtn)
                debugBtn.snp.makeConstraints { make in
                    make.leading.equalTo(10)
                    make.centerY.equalToSuperview()
                }
            }
#endif
        }
}
// 首页
class HomeView: UIView {
    weak var delegate: HomeViewDelegate?
    let bgView = GradientView()
    let nav = HomeNavView()
    let scroll = UIScrollView()
    let content = UIStackView()
    let textView = HomeInputTextView()
```

```
        let actionsBtnView = UIView()
        var quanWangBiJia = HomeActionButton(style: .large)
        var aiKanJia = HomeActionButton(style: .normal)
        let bottomView = HomeBottomView()
        let productItem = HomeProductItemView()
        let guideView = HomeGuideView()
        var bottomMaxCons: Constraint?
        override init(frame: CGRect) {
            super.init(frame: frame)
            setupUI()
        }
        required init?(coder: NSCoder) {
            fatalError("init(coder:) has not been implemented")
        }
        func setupUI() {
            addSubview(bgView)
            bgView.snp.makeConstraints { make in
                make.edges.equalToSuperview()
            }
            bgView.gradientInfo = GradientInfo(colors: ["#7ABDFF".uicolor().cgColor,"#E6F4FE".uicolor().cgColor], startPoint: CGPoint(x: 0.5, y: 0), endPoint: CGPoint(x: 0.5, y: 0.4))
            let bg1Image = UIImageView(image: UIImage(named: "home_v2_bg1"))
            bg1Image.contentMode = .scaleAspectFit
            bgView.addSubview(bg1Image)
            bg1Image.snp.makeConstraints { make in
                make.width.equalToSuperview()
                make.height.equalTo(bg1Image.snp.width).multipliedBy(576.0/750.0)
            }
            addSubview(nav)
            nav.snp.makeConstraints { make in
                make.leading.top.trailing.equalToSuperview()
            }
            nav.mineBtn.addTarget(self, action: #selector(clickMineBtn), for: .touchUpInside)
            scroll.showsVerticalScrollIndicator = false
            scroll.showsHorizontalScrollIndicator = false
            scroll.delegate = self
            addSubview(scroll)
            scroll.snp.makeConstraints { make in
                make.leading.trailing.bottom.equalToSuperview()
                make.top.equalTo(nav.snp.bottom)
                make.width.equalToSuperview()
            }
            let container = UIView()
            scroll.addSubview(container)
            container.snp.makeConstraints { make in
                make.edges.equalToSuperview()
                make.width.equalToSuperview()
                make.height.greaterThanOrEqualToSuperview()
            }
//            content.distribution = .fill
```

```
        content.axis = .vertical
        content.alignment = .center
        container.addSubview(content)
        content.snp.makeConstraints { make in
            make.leading.trailing.equalToSuperview()
            make.width.equalToSuperview()
            make.top.equalTo(60)
        }
        content.setContentHuggingPriority(.required, for: .vertical)
        content.setContentCompressionResistancePriority(.required, for: .vertical)
        let bgTitle = UIImageView(image: UIImage(named: "home_title"))
        bgTitle.snp.makeConstraints { make in
            make.size.equalTo(CGSize(width: 285, height: 42))
        }
        content.addArrangedSubview(bgTitle)
        content.setCustomSpacing(60, after: bgTitle)
        let guideLabel = UILabel()
        guideLabel.setContentHuggingPriority(.required, for: .vertical)
        guideLabel.setContentCompressionResistancePriority(.required, for: .vertical)
        content.addArrangedSubview(guideLabel)
        guideLabel.text = "请粘贴商品链接/输入商品名称："
        guideLabel.textColor = "#0059FF".uicolor()
        guideLabel.font = .PingFang(ofSize: 16, type: .semibold)
        guideLabel.numberOfLines = 1
        content.setCustomSpacing(5, after: guideLabel)
        content.addArrangedSubview(textView)
        textView.snp.makeConstraints { make in
            make.leading.equalTo(self).offset(16)
            make.trailing.equalTo(self).offset(-16)
            make.leading.equalTo(guideLabel).offset(-15)
        }
        textView.delegate = self
        textView.input.delegate = self
        content.setCustomSpacing(15, after: textView)
        content.addArrangedSubview(productItem)
        content.setCustomSpacing(15, after: productItem)
        productItem.snp.makeConstraints { make in
            make.leading.trailing.equalTo(textView)
            make.height.equalTo(110)
        }
        productItem.isHidden = true
        content.addArrangedSubview(actionsBtnView)
        actionsBtnView.snp.makeConstraints { make in
            make.leading.trailing.equalTo(self)
            make.height.equalTo(40)
        }
        quanWangBiJia.title.text = "全网比价"
        quanWangBiJia.btn.addTarget(self, action: #selector(clickQuanWangBiJia),
for: .touchUpInside)
        aiKanJia.title.text = "砍价"
        aiKanJia.icon.image = UIImage(named: "home_action_ai_icon")
```

```
            aiKanJia.icon.isHidden = false
            aiKanJia.btn.addTarget(self, action: #selector(clickAIKanJia), for: .touchUpInside)
            aiKanJia.isHidden = true
            actionsBtnView.addSubview(quanWangBiJia)
            actionsBtnView.addSubview(aiKanJia)
            quanWangBiJia.snp.makeConstraints { make in
                make.leading.equalTo(40)
                make.top.equalToSuperview()
            }
            aiKanJia.snp.makeConstraints { make in
                make.trailing.equalTo(-40)
                make.top.equalToSuperview()
            }
            content.addArrangedSubview(guideView)
            guideView.snp.makeConstraints { make in
                make.width.equalToSuperview()
                make.centerX.equalToSuperview()
            }
            container.addSubview(bottomView)
            bottomView.setContentHuggingPriority(.required, for: .vertical)
            bottomView.setContentCompressionResistancePriority(.required, for: .vertical)
            bottomView.snp.makeConstraints { make in
                make.leading.trailing.equalToSuperview()
                make.bottom.equalToSuperview()
                self.bottomMaxCons                                                    =
make.top.greaterThanOrEqualTo(content.snp.bottom).offset(30).constraint
            }
            let cancelBtn = UIButton()
            container.addSubview(cancelBtn)
            cancelBtn.snp.makeConstraints { make in
                make.leading.trailing.equalToSuperview()
                make.top.equalTo(container)
                make.bottom.equalTo(guideLabel.snp.top)
            }
            cancelBtn.addTarget(self, action: #selector(clickCancelEdit), for: .touchUpInside)
        }
        @objc func clickCancelEdit() {
            endEditing(true)
        }
        func addQuanWangBiJia() {
            let btn = UIButton()
            addSubview(btn)
        }
        @objc func clickMineBtn() {
            endEditing(true)
            delegate?.homeView(homeView: self, didClickMineButton: nav.mineBtn)
        }
        var inputText: String? {
            textView.input.text
        }
        func updateBottom(_ bottom: InitBottomBanner?) {
```

```
        bottomView.updateImages(images: bottom?.items?.compactMap({$0.image?.url}))
    }
    func showProduct() {
        productItem.isHidden = false
        UIViewPropertyAnimator.runningPropertyAnimator(withDuration: 0.3, delay: 0) {
            self.productItem.alpha = 1.0
            self.productItem.isHidden = false
        }
    }
    func dismissProduct() {
        guard !productItem.isHidden else {
            return
        }
        UIViewPropertyAnimator.runningPropertyAnimator(withDuration: 0.3, delay: 0) {
            self.productItem.alpha = 0.0
            self.productItem.isHidden = true
        }
    }
    @objc
    func clickQuanWangBiJia() {
        endEditing(true)
        delegate?.homeView(homeView: self, clickBiJia: quanWangBiJia)
    }
    @objc
    func clickAIKanJia() {
        endEditing(true)
        delegate?.homeView(homeView: self, clickKanJia: aiKanJia)
    }
    // MARK: - Public
    /// 更新首页数据
    /// - Parameter data: v1/init 数据
    public func updateInitData(data: InitResponseData?) {
        guard let data = data else { return }
        nav.mineIcon.isHighlighted = data.notification?.center ?? 0 > 0 // 更新个人中心 icon
的小红点
        textView.updatePlaceholder(data.inputHint) // 更新输入框提示语
        if let config = data.actionButtonConfig { // 处理按钮是否展示
            switch config {
            case .onlyQuanWangBiJia:
                aiKanJia.isHidden = true
                quanWangBiJia.style = .large
            case .bijiaAndKanJia:
                aiKanJia.isHidden = false
                quanWangBiJia.style = .normal
            }
        }
        updateBottom(data.bottomBanner) // 更新轮播图数据
    }
    public func updatePasteData(data: PasteData?) {
        guard let data = data else {return}
        aiKanJia.enable = data.kanJiaEnable ?? false
```

```
            quanWangBiJia.enable = data.quanWangBiJiaEnable ?? false
            if let product = data.product {
                productItem.productData = product
                if productItem.isHidden {
                    showProduct()
                }
            } else {
                productItem.productData = nil
                if !productItem.isHidden {
                    dismissProduct()
                }
            }
        }
        public func resetInput() {
            dismissProduct()
            aiKanJia.enable = true
            quanWangBiJia.enable = true
        }
}
extension HomeView: HomeInputTextViewDelegate, GrowingTextViewDelegate {
        func textView(textView: HomeInputTextView, textDidChanged input: GrowingTextView) {
            delegate?.homeView(homeView: self, textDidChanged: input)
        }
        @objc func growingTextViewShouldBeginEditing(_ growingTextView: GrowingTextView) ->
Bool {
            delegate?.homeView(homeView: self, shouldTextViewEditing: growingTextView) ??
true
        }
        @objc func growingTextViewDidChange(_ growingTextView: GrowingTextView) {
            textView.textDidChange()
        }
        @objc func growingTextView(_ growingTextView: GrowingTextView, willChangeHeight
height: CGFloat, difference: CGFloat) {
            textView.inputHeightCons?.update(offset: max(height, 36.0))
        }
        @objc func growingTextViewShouldReturn(_ growingTextView: GrowingTextView) ->
Bool {
            delegate?.homeView(homeView: self, shouldTextViewReturn: growingTextView) ??
true
        }
}
extension HomeView: UIScrollViewDelegate {
        func scrollViewWillBeginDragging(_ scrollView: UIScrollView) {
            endEditing(true)
        }
}
class HomeActionButton: UIView {
        enum Style: Int {
            case normal
            case large
        }
```

```
let bgView = UIView()
let icon = UIImageView()
let title = UILabel()
let btn = UIButton()
// 阴影
let shadow = CAShapeLayer()
var style: Style {
    didSet {
        if oldValue != style {
            updateStyle()
        }
    }
}
var enable: Bool = true {
    didSet {
        isUserInteractionEnabled = enable
        bgView.alpha = enable ? 1.0 : 0.5
    }
}
init(style: Style) {
    self.style = style
    super.init(frame: .init(origin: .zero, size: Self.styleSize(style)))
    // 阴影
    shadow.shadowColor = UIColor(red: 0, green: 0.44, blue: 1, alpha: 0.2).cgColor
    shadow.shadowOffset = CGSize(width: 0, height: 5)
    shadow.shadowOpacity = 1
    shadow.shadowRadius = 5
    shadow.fillColor = UIColor.clear.cgColor
    layer.addSublayer(shadow)
    addSubview(bgView)
    bgView.layer.cornerRadius = 20
    bgView.backgroundColor = "#0059FF".uicolor()
    bgView.snp.makeConstraints { make in
        make.size.equalTo(Self.styleSize(style))
        make.edges.equalToSuperview()
    }
    updateStyle()
    icon.frame = CGRect(origin: .zero, size: CGSize(width: 28, height: 23))// icon size
    icon.isHidden = true
    title.textColor = .white
    title.font = .PingFang(ofSize: 16, type: .semibold)
    let stack = UIStackView(arrangedSubviews: [
        icon,
        title
    ])
    stack.axis = .horizontal
    stack.spacing = 3.0
    bgView.addSubview(stack)
    stack.snp.makeConstraints { make in
        make.center.equalToSuperview()
    }
```

```
            addSubview(btn)
            btn.snp.makeConstraints { make in
                make.edges.equalToSuperview()
            }
        }
        required init?(coder: NSCoder) {
            fatalError("init(coder:) has not been implemented")
        }
        static func styleSize(_ style: Style) -> CGSize {
            var width: CGFloat
            switch style {
            case .normal:
                width = floor((DeviceManager.shared.screenSize.width - 40 - 40 - 10) * 0.5)
            case .large:
                width = floor((DeviceManager.shared.screenSize.width - 45 - 45))
            }
            let size = CGSize(width: width, height: 40.0)
            return size
        }
        func updateStyle() {
            let size = Self.styleSize(style)
            let rect = CGRect(origin: .zero, size: size)
            shadow.frame = CGRect(origin: .zero, size: size)
            shadow.path = UIBezierPath(roundedRect: rect, cornerRadius: 20).cgPath
            bgView.snp.remakeConstraints { make in
                make.size.equalTo(Self.styleSize(style))
                make.edges.equalToSuperview()
            }
        }
}
protocol HomeInputTextViewDelegate: AnyObject {
        func textView(textView: HomeInputTextView, textDidChanged input: GrowingTextView)
}
class HomeInputTextView: UIView {
        let bgView = UIView()
        let input = GrowingTextView()
        let clearBtn = UIButton()
        let shadowLayer = CAShapeLayer()
        weak var delegate: HomeInputTextViewDelegate?
        var inputHeightCons: Constraint?
        override init(frame: CGRect) {
            super.init(frame: frame)
            setContentHuggingPriority(.required, for: .vertical)
            setContentCompressionResistancePriority(.required, for: .vertical)
            shadowLayer.frame = CGRect(origin: .zero, size: frame.size)
            shadowLayer.path = UIBezierPath(roundedRect: shadowLayer.frame, cornerRadius:
15).cgPath
            shadowLayer.shadowColor = UIColor(red: 0.03, green: 0.42, blue: 1, alpha:
0.02).cgColor
            shadowLayer.shadowOffset = CGSize(width: 0, height: 3.5)
            shadowLayer.shadowOpacity = 1
```

```
            shadowLayer.shadowRadius = 2
            shadowLayer.fillColor = UIColor(white: 1, alpha: 0.9).cgColor
            layer.addSublayer(shadowLayer)
            let bg = UIView()
            bg.backgroundColor = UIColor(white: 1, alpha: 0.9)
            addSubview(bg)
            bg.snp.makeConstraints { make in
                make.edges.equalToSuperview()
            }
            bg.layer.borderWidth = 2
            bg.layer.borderColor = "#0059FF".uicolor().cgColor
            bg.layer.cornerRadius = 15
            setupInput()
            clearBtn.setImage(UIImage(named: "home_input_clear"), for: .normal)
            clearBtn.setImage(UIImage(named: "home_input_clear"), for: .highlighted)
            clearBtn.isHidden = true
            addSubview(clearBtn)
            clearBtn.snp.makeConstraints { make in
                make.top.equalTo(10)
                make.size.equalTo(CGSize(width: 40, height: 40))
                make.trailing.equalToSuperview()
            }
            clearBtn.addTarget(self, action: #selector(clickClear), for: .touchUpInside)
        }
        required init?(coder: NSCoder) {
            fatalError("init(coder:) has not been implemented")
        }
        override func layoutSubviews() {
            super.layoutSubviews()
            if shadowLayer.frame.size != frame.size {
                shadowLayer.frame = CGRect(origin: .zero, size: frame.size)
                shadowLayer.path = UIBezierPath(roundedRect: shadowLayer.frame, cornerRadius:
15).cgPath
            }
        }
        func setupInput() {
            let textMaxWidth = DeviceManager.shared.screenSize.width - 16 - 16 - 15 - 40
            input.frame = CGRect(x: 15, y: 12, width: textMaxWidth, height: 36)
            addSubview(input)
            input.snp.makeConstraints { make in
                make.leading.equalTo(15)
                make.top.equalTo(12)
                make.width.equalTo(textMaxWidth)
                make.bottom.equalTo(-12)
                self.inputHeightCons = make.height.equalTo(36).constraint
            }
            input.setContentHuggingPriority(.required, for: .vertical)
            input.setContentCompressionResistancePriority(.required, for: .vertical)
            input.font = .PingFang(ofSize: 14)
            input.backgroundColor = UIColor.clear
            input.textColor = "#232323".uicolor()
```

```
            input.isOpaque = false
            input.clipsToBounds = true
            input.returnKeyType = .done
            input.internalTextView.backgroundColor = UIColor(white: 1.0, alpha: 0.9)
            input.internalTextView.isOpaque = false
            input.internalTextView.contentMode = .left
            input.internalTextView.enablesReturnKeyAutomatically = true
            //input.internalTextView.scrollIndicatorInsets = UIEdgeInsets(top: 0, left: 0, bottom: 5-0.5, right: 0)
            updatePlaceholder(MainDataManager.shared.initData?.inputHint)
            input.maxNumberOfLines = 4
            let bar = UIToolbar()
            let sapce = UIBarButtonItem(barButtonSystemItem: .flexibleSpace, target: nil, action: nil)
            let done = UIBarButtonItem(title: "完成", style: .done, target: self, action: #selector(keyboardDone))
            bar.items = [sapce,done]
            bar.sizeToFit()
            input.internalTextView.inputAccessoryView = bar
        }
        @objc func keyboardDone() {
            let _ = input.resignFirstResponder()
        }
        @objc func keyboardPaste() {
            let _ = input.resignFirstResponder()
            input.text = PasteboardManager.shared.getPasteboard()
            textDidChange()
        }
        @objc func clickClear() {
            input.text = ""
            textDidChange()
        }
        func textDidChange() {
            delegate?.textView(textView: self, textDidChanged: input)
            if input.hasText { // 有内容展示清空按钮
                clearBtn.isHidden = false
            } else {
                clearBtn.isHidden = true
            }
        }
        func updatePlaceholder(_ placeholder: String?) {
            if let placeholder = placeholder {
                input.isPlaceholderEnabled = true
                input.placeholder = NSAttributedString(string: placeholder, attributes: [.foregroundColor:"#999999".uicolor(),.font:UIFont.PingFang(ofSize: 14)])
            } else {
                input.isPlaceholderEnabled = false
            }
        }
}
class HomeBottomView: UIView {
```

```
                make.centerX.equalToSuperview()
            }
            let btn = UIButton()
            lastItem.addSubview(btn)
            btn.snp.makeConstraints { make in
                make.edges.equalToSuperview()
            }
            btn.addTarget(self, action: #selector(clickBtn), for: .touchUpInside)
            reloadItem([
                WelcomeItemView(imageName: "welcome1_v1"),
                WelcomeItemView(imageName: "welcome2_v1"),
                lastItem
                        ])
        }
        override func handle(paramaters: [String : String]?) {
            super.handle(paramaters: paramaters)
            nonav = true
            noback = true
            nologin = true
        }
        @objc func clickBtn() {
            delegate?.welcomeFinished(welcome: self)
        }
        func reloadItem(_ items: [WelcomeItemView]) {
            hStack.subviews.forEach({$0.removeFromSuperview()})
            items.forEach({hStack.addArrangedSubview($0)})
            items.forEach { itemView in
                hStack.addArrangedSubview(itemView)
                itemView.snp.makeConstraints { make in
                    make.width.equalTo(view)
                    make.height.equalTo(view)
                }
            }
        }
        func scrollViewDidScroll(_ scrollView: UIScrollView) {
            let page = Int(scrollView.contentOffset.x / scrollView.frame.size.width) // 通过scrollView 内容的偏移计算当前显示的是第几页
            pageControl.currentPage = page //设置 pageController 的当前页
//            let lastWidth = CGFloat(2) * scrollView.frame.size.width
//            if(scrollView.contentOffset.x > lastWidth)//如果在最后一个页面继续滑动的话就会跳转到主页面
//            {
//                delegate?.welcomeFinished(welcome: self)
//            }
        }
        @objc func pageAction(_ sender: UIPageControl) {
            let page = sender.currentPage
            var frame = scrollView.frame
            frame.origin.x = frame.size.width * CGFloat(page)
            frame.origin.y = 0
            print("pageAction:\(page), frame:\(frame)")
```

```
            scrollView.scrollRectToVisible(frame, animated:true)
        }
}
class WelcomeItemView: UIView {
        let imageView = UIImageView()
        required init?(coder: NSCoder) {
            fatalError("init(coder:) has not been implemented")
        }
        init(imageName: String) {
            super.init(frame: .zero)
            translatesAutoresizingMaskIntoConstraints = false
            addSubview(imageView)
            imageView.snp.makeConstraints { make in
                make.edges.equalToSuperview()
            }
            imageView.image = UIImage(named: imageName)
            imageView.contentMode = .scaleAspectFill
        }
}
import UIKit
class LoginController: WebBaseController {
        var didLoginCallback: ((Bool)->Void)?
        init(completion: ((Bool) -> Void)? = nil) {
            didLoginCallback = completion
            super.init(querys: nil, url: URL(string: loginPath)!, userScripts: nil)
            NotificationCenter.default.addObserver(self, selector: #selector(didLogin(notification:)),
name: .login, object: nil)
        }
        required init?(coder: NSCoder) {
            fatalError("init(coder:) has not been implemented")
        }
        override func viewDidLoad() {
            super.viewDidLoad()
        }
        deinit {
            NotificationCenter.default.removeObserver(self)
        }
        override func handle(paramaters: [String : String]?) {
            super.handle(paramaters: paramaters)
            nologin = true
            nonav = true
            noSwipeBack = true
        }
        @objc func didLogin(notification: Notification) {
            PageManager.shared.popViewController(animated: false) {[weak self] in
                DispatchQueue.main.async {
                    self?.didLoginCallback?(User.shared.didLogin)
                }
            }
        }
        @discardableResult
```

```
    static func tryShowLogin() -> Bool {
        guard !User.shared.didLogin else {
            return false
        }
        if let _ = PageManager.shared.rootNavigationController()?.topViewController as? LoginController {
            return false // 如果当前的 top 就是 login，则没必要反复弹 login
        }
        DispatchQueue.main.asyncAfter(deadline: .now()+0.1) {
            let login = LoginController(completion: nil)
            PageManager.shared.open(viewController: login, style: .push, animated: false)
        }
        return true
    }
    static func tryGetLogin() -> LoginController? {
        guard !User.shared.didLogin else {
            return nil
        }
        if MainDataManager.shared.isGuest {
            return nil
        }
        return LoginController(completion: nil)
    }
}
import UIKit
import SnapKit
protocol LaunchControllerDelegate: AnyObject {
    func launchDidFinish(launch:LaunchController)
}
struct LaunchTimeState: OptionSet {
    let rawValue: Int
    static let timeout = LaunchTimeState(rawValue: (1 << 0))
    //static let main = LaunchTimeState(rawValue: (1 << 1))
    static let config = LaunchTimeState(rawValue: (1 << 2))
}
enum LaunchState {
    case none
    case finishLaunch
    case finishGuidance
}
class LaunchView: UIView {
    override init(frame: CGRect) {
        super.init(frame: frame)
        backgroundColor = .white
        let imageView = UIImageView(image: UIImage(named: "splash_v3@3x"))
        addSubview(imageView)
        imageView.snp.makeConstraints { make in
            make.size.equalTo(CGSize(width: 140, height: 45))
            make.centerX.equalTo(self.safeAreaLayoutGuide.snp.centerX)
            make.centerY.equalTo(self.safeAreaLayoutGuide.snp.centerY).offset(-140)
        }
```

```
    }
    required init?(coder: NSCoder) {
        fatalError("init(coder:) has not been implemented")
    }
}
class LaunchController: BaseController {
    weak var delegate: LaunchControllerDelegate?
    var launchTime: LaunchTimeState = [] {
        didSet {
            if launchTime != [] {
                refreshState()
            }
        }
    }
    var state: LaunchState = .none {
        didSet {
            switch state {
            case .none:
                break
            case .finishLaunch:
                finishLaunch()
            case .finishGuidance:
                finishGuidance()
            }
        }
    }
    let launchView = LaunchView()
    override func loadView() {
        view = launchView
    }
    override func viewDidLoad() {
        super.viewDidLoad()
        view.backgroundColor = .white
        NotificationCenter.default.addObserver(self, selector:
#selector(configFinish(notification:)), name: .configFinish, object: nil)
    }
    deinit {
        NotificationCenter.default.removeObserver(self)
    }
    override func handle(paramaters: [String : String]?) {
        super.handle(paramaters: paramaters)
        nonav = true
    }
    func start() {
        perform(#selector(requestTimeout), with: nil, afterDelay: 5)
        requestConfig()
    }
    func cancel() {
        cancelConfig()
    }
    func requestConfig() {
```

```
            MainDataManager.shared.requestConfig()
        }
        func cancelConfig() {
            MainDataManager.shared.cancelConfig()
        }
        @objc private func configFinish(notification: Notification) {
            guard let result = notification.userInfo?["result"] as? Bool, result else { return }
            launchTime.insert(.config)
        }
        // MARK: - private
        @objc private func requestTimeout() {
            launchTime.insert(.timeout)
        }
        func refreshState() {
            guard launchTime.contains(.timeout) || launchTime.contains([.config]) else {return}
            launchTime = []
            NSObject.cancelPreviousPerformRequests(withTarget: self)
            cancel()
            state = .finishGuidance
        }
        func finishLaunch() {
            delegate?.launchDidFinish(launch: self)
        }
        func finishGuidance() {
            //  当前不需要
            state = .finishLaunch
        }
}
import UIKit
import WebKit
import SnapKit
class PrivacyController: BaseController {
        var finishCompletion: (()->Void)?
        var disagreeAlertView: UIView?
        var privateAlert = UIView()
        override func handle(paramaters: [String : String]?) {
            super.handle(paramaters: paramaters)
            nonav = true
            noback = true
            nologin = true
        }
        init(complete: (() -> Void)?) {
            self.finishCompletion = complete
            super.init(querys: nil)
        }
        required init?(coder: NSCoder) {
            fatalError("init(coder:) has not been implemented")
        }
        override func loadView() {
            view = LaunchView()
        }
```

```
override func viewDidLoad() {
    super.viewDidLoad()
    self.title =    Utils.displayName()+"协议"
    navView.title = self.title
    setupUI()
}
override func viewWillAppear(_ animated: Bool) {
    super.viewWillAppear(animated)
    Track.event("display_privacy")
}
func setupUI() {
    let bgView = UIView()
    bgView.backgroundColor = UIColor(white: 0, alpha: 0.5)
    view.addSubview(bgView)
    bgView.snp.makeConstraints { make in
        make.edges.equalToSuperview()
    }
    setupDisagreeView()
    disagreeAlertView?.isHidden = true
    setupPrivateAlertView()
}
func setupPrivateAlertView() {
    view.addSubview(privateAlert)
    privateAlert.snp.makeConstraints { make in
        make.left.equalTo(42)
        make.right.equalTo(-42)
        make.centerY.equalToSuperview().offset(10)
    }
    // 背景
    let bgView = UIView()
    bgView.backgroundColor = .white
    bgView.layer.cornerRadius = 10
    privateAlert.addSubview(bgView)
    bgView.snp.makeConstraints { make in
        make.edges.equalToSuperview()
    }
    let title = UILabel()
    title.font = .PingFang(ofSize: 14, type: .medium)
    title.textColor = "#222222".uicolor()
    title.text = "\(Utils.displayName())协议"
    privateAlert.addSubview(title)
    title.snp.makeConstraints { make in
        make.centerX.equalToSuperview()
        make.top.equalTo(12)
    }
    // 隐私内容
    let webView = WKWebView(frame: .zero)
    privateAlert.addSubview(webView)
    webView.scrollView.bounces = false
    webView.scrollView.showsVerticalScrollIndicator = false
    webView.scrollView.showsHorizontalScrollIndicator = false
```

```
        webView.snp.makeConstraints { make in
            make.left.equalTo(20)
            make.right.equalTo(-20)
            make.height.equalTo(164)
            make.top.equalTo(title.snp.bottom).offset(10)
        }
        if let path = Bundle.main.path(forResource: "privacy_content", ofType: "html") {
            webView.load(URLRequest(url: URL(fileURLWithPath: path)))
        }
        // 隐私政策
        let agreementTextView = UITextView(frame: .zero)
        privateAlert.addSubview(agreementTextView)
        agreementTextView.backgroundColor = .clear;
        agreementTextView.isEditable = false;
        agreementTextView.isScrollEnabled = false
        agreementTextView.delegate = self;
        agreementTextView.textContainer.lineFragmentPadding = 0.0;
        agreementTextView.textContainerInset = .zero;
        agreementTextView.linkTextAttributes = [.foregroundColor:"#0059FF".uicolor()]
        agreementTextView.attributedText = getBottomArttributedString()
        agreementTextView.snp.makeConstraints { make in
            make.top.equalTo(webView.snp.bottom).offset(15);
            make.left.right.equalTo(webView)
            make.height.equalTo(70);
        }
        // 同意按钮
        let agreeButton = UIButton(frame: .zero)
        agreeButton.setTitle("同意", for: .normal)
        agreeButton.setTitleColor(.white, for: .normal)
        agreeButton.titleLabel?.font = UIFont.PingFang(ofSize: 16, type: .medium)
        agreeButton.backgroundColor = "#0059FF".uicolor()
        agreeButton.layer.cornerRadius = 19.0
        agreeButton.clipsToBounds = true
        agreeButton.addTarget(self, action: #selector(clickAgreeBtn(sender:)),
for: .touchUpInside)
        privateAlert.addSubview(agreeButton)
        agreeButton.snp.makeConstraints { make in
            make.left.equalTo(28)
            make.right.equalTo(-28)
            make.height.equalTo(38);
            make.top.equalTo(agreementTextView.snp.bottom).offset(2)
        }
        // 不同意按钮
        let disagreeButton = UIButton(frame: .zero)
        disagreeButton.setTitle("不同意", for: .normal)
        disagreeButton.setTitleColor("0x999999".uicolor(), for: .normal)
        disagreeButton.titleLabel?.font = UIFont.PingFang(ofSize: 12)
        disagreeButton.backgroundColor = .white
        disagreeButton.addTarget(self, action: #selector(clickDisagreeBtn(sender:)),
for: .touchUpInside)
        privateAlert.addSubview(disagreeButton)
```

```
            disagreeButton.snp.makeConstraints { make in
                make.centerX.equalToSuperview()
                make.bottom.equalToSuperview().offset(-22)
                make.top.equalTo(agreeButton.snp.bottom).offset(18)
            }
        }
        func getBottomArttributedString() -> NSAttributedString {
            let paragraphStyle = NSMutableParagraphStyle()
            paragraphStyle.lineSpacing = 4
            let str1 = "《用户使用协议》"
            let str2 = "《隐私政策》"
            let agreementArttributedString = NSMutableAttributedString(string: "如您同意\(str1)、
\(str2)，请点击“同意”开始使用我们的产品和服务，我们尽全力保护您的个人信息安全",
                                                                       attributes:
[.font:UIFont.PingFang(ofSize: 12),
.foregroundColor:"0x666666".uicolor(),
.paragraphStyle:paragraphStyle])
            let content = agreementArttributedString.string
            if let range = content.getNSRange(str1) {
                agreementArttributedString.addAttribute(.link, value: URL(string: "register://")!,
range: range)
            }
            if let range = content.getNSRange(str2) {
                agreementArttributedString.addAttribute(.link, value: URL(string: "private://")!,
range: range)
            }
            return agreementArttributedString
        }
        func getAlertArttributedString() -> NSAttributedString {
            let paragraphStyle = NSMutableParagraphStyle()
            paragraphStyle.lineSpacing = 4
            let str1 = "《用户使用协议》"
            let str2 = "《隐私政策》"
            let agreementArttributedString = NSMutableAttributedString(string: "您可再次查看
\(str1)、\(str2)全文。如您同意我的政策内容后，您可继续使用。",
                                                                       attributes:
[.font:UIFont.PingFang(ofSize: 12),
.foregroundColor:"0x666666".uicolor(),
.paragraphStyle:paragraphStyle])
            let content = agreementArttributedString.string
            if let range = content.getNSRange(str1) {
                agreementArttributedString.addAttribute(.link, value: URL(string: "register://")!,
range: range)
            }
            if let range = content.getNSRange(str2) {
                agreementArttributedString.addAttribute(.link, value: URL(string: "private://")!,
range: range)
            }
            return agreementArttributedString
        }
        func setupDisagreeView() {
```

```
        let container = UIView(frame: .zero)
        view.addSubview(container)
        container.snp.makeConstraints { make in
            make.left.equalTo(42)
            make.right.equalTo(-42)
            make.centerY.equalToSuperview().offset(10)
        }
        let bgView = UIView()
        bgView.backgroundColor = .white
        bgView.layer.cornerRadius = 10
        container.addSubview(bgView)
        bgView.snp.makeConstraints { make in
            make.edges.equalTo(container)
        }
        let titleLabel = UILabel()
        titleLabel.textColor = "#222222".uicolor()
        titleLabel.text = "我们将充分尊重并保护您的隐私请您放心"
        titleLabel.font = .PingFang(ofSize: 14, type: .medium)
        titleLabel.textAlignment = .center
        titleLabel.numberOfLines = 0
        container.addSubview(titleLabel)
        titleLabel.snp.makeConstraints { make in
            make.left.equalTo(20)
            make.right.equalTo(-20)
            make.top.equalTo(20)
        }
        let agreementTextView = UITextView(frame: .zero)
        container.addSubview(agreementTextView)
        agreementTextView.backgroundColor = .clear;
        agreementTextView.isEditable = false;
        agreementTextView.isScrollEnabled = false
        agreementTextView.delegate = self;
        agreementTextView.textContainer.lineFragmentPadding = 0.0;
        agreementTextView.textContainerInset = .zero;
        agreementTextView.linkTextAttributes = [.foregroundColor:"#0059FF".uicolor()]
        agreementTextView.attributedText = getBottomArttributedString()
        agreementTextView.snp.makeConstraints { make in
            make.left.equalTo(20)
            make.right.equalTo(-20)
            make.height.equalTo(70)
            make.top.equalTo(titleLabel.snp.bottom).offset(15)
        }
        let agreeButton = UIButton(frame: .zero)
        agreeButton.setTitle("我知道了", for: .normal)
        agreeButton.setTitleColor(.white, for: .normal)
        agreeButton.titleLabel?.font = UIFont.PingFang(ofSize: 16, type: .medium)
        agreeButton.backgroundColor = "#0059FF".uicolor()
        agreeButton.layer.cornerRadius = 19.0
        agreeButton.clipsToBounds = true
        agreeButton.addTarget(self,          action:          #selector(confirmButtonClicked(_:)),
for: .touchUpInside)
```

```
            container.addSubview(agreeButton)
            agreeButton.snp.makeConstraints { make in
                make.left.equalTo(28)
                make.right.equalTo(-28)
                make.height.equalTo(38);
                make.top.equalTo(agreementTextView.snp.bottom).offset(10)
                make.bottom.equalTo(-25)
            }
            disagreeAlertView = container
        }
        @objc func clickAgreeBtn(sender:UIButton) {
            Track.btnClick(btnName: "privacy_agree")
            self.finishCompletion?()
        }
        @objc func clickDisagreeBtn(sender:UIButton) {
            Track.btnClick(btnName: "privacy_disagree")
            showAlert()
        }
        func showAlert() {
            privateAlert.isHidden = true
            disagreeAlertView?.isHidden = false
            Track.event("display_privacy_rationale")
        }
        @objc func confirmButtonClicked(_ sender: UIButton) {
            disagreeAlertView?.isHidden = true
            privateAlert.isHidden = false
            Track.btnClick(btnName: "privacy_rationale_know")
        }
}
extension PrivacyController: UITextViewDelegate {
        func textView(_ textView: UITextView, shouldInteractWith URL: URL, in characterRange:
NSRange, interaction: UITextItemInteraction) -> Bool {
            return shouldInteractWithTextView(textView,shouldInteractWith:URL)
        }
        func shouldInteractWithTextView(_ textView: UITextView, shouldInteractWith URL: URL) ->
Bool {
            guard let scheme = URL.scheme else { return true }
            var linkUrl: URL?
            if scheme == "private" {
                guard let path = Bundle.main.path(forResource: "privacy_policy", ofType: "html")
else { return true}
                linkUrl = NSURL(fileURLWithPath: path) as URL
            } else if scheme == "register" {
                guard let path = Bundle.main.path(forResource: "reg_agree", ofType: "html") else
{ return true}
                linkUrl = NSURL(fileURLWithPath: path) as URL
            }
            guard let linkUrl = linkUrl else { return true }
            let vc = CommonWebViewController(querys: nil, url: linkUrl)
            vc.requireGo = false
            vc.nologin = true
```

```
            PageManager.shared.open(viewController: vc)
            return false
        }
}
import Foundation
import Moya
import Alamofire
protocol CodableEnumeration: RawRepresentable, Codable where RawValue: Codable {
    static var defaultCase: Self { get }
}
extension CodableEnumeration {
    init(from decoder: Decoder) throws {
        let container = try decoder.singleValueContainer()
        do {
            let decoded = try container.decode(RawValue.self)
            self = Self.init(rawValue: decoded) ?? Self.defaultCase
        } catch {
            self = Self.defaultCase
        }
    }
}
enum NetAPI {
    case registerDevice([String:String],[String:String])
    case updateDevice([String:String],[String:String])
    case renew
    case ud(String, String?)
    case authTaoke(TaokeAuthInfo)
    case v1Init
    case appConfig
    case v1Dispatch(DispatchType,String,String?)
    case v1Clipboard(String)
    case appleAd([String:String], AAAType)
    case paste(String)
    case slog(Int, [String: Any]?) //rt,public query
    case notificationIncrement
    case start
}
extension NetAPI: TargetType {
    var baseURL: URL {
        switch self {
        case .ud(_, _):
            return URL(string: "https://gw.ruyisheng.com")!
        case .slog(_, _):
            return URL(string: "https://api.ruyisheng.com")!
        default:
            return URL(string: "https://"+phpHostStr)!
        }
    }
    var path: String {
        switch self {
        case .registerDevice(_,_):
```

```
                return "/passport/service/regDev"
            case .updateDevice(_,_):
                return "/passport/service/updateDev"
            case .renew:
                return "/gateway/common/user/renewal.htm"
            case .ud(_, _):
                return "/app/v2/service/ud.htm"
            case .authTaoke(_):
                return "/gateway/app/v1/auth/taoke.htm"
            case .v1Init:
                return "/gateway/common/v1/init.htm"
            case .appConfig:
                return "/gateway/common/v1/appconfig.htm"
            case .v1Dispatch(_,_,_):
                return "/gateway/common/v1/dispatch.htm"
            case .v1Clipboard:
                return "/gateway/common/v1/clipboard.htm"
            case .appleAd(_,_):
                return "/gateway/common/v1/apple/ad.htm"
            case .paste(_):
                return "/gateway/common/v1/paste.htm"
            case .slog(_, _):
                return "/app/s/log.htm"
            case .notificationIncrement:
                return "/gateway/common/v1/notification/increment.htm"
            case .start:
                return "/gateway/common/v1/start.htm"
            }
        }
        var method: Moya.Method {
            switch self {
case .registerDevice(_,_), .updateDevice(_,_),.authTaoke(_),.v1Dispatch(_,_,_),.v1Clipboard(_),.a
ppleAd(_, _),.paste(_),.notificationIncrement:
                return .post
            default:
                return .get
            }
        }
        var sampleData: Data {
            return "{}".data(using: String.Encoding.utf8)!
        }
        var task: Moya.Task {
            // 公共参数
            var publicParams: [String: Any] = ["source":sourceStr,"version":Utils.marketVersion()]
            publicParams["mc"] = channel
            if let uid = User.shared.userId {
                publicParams["uid"] = uid
            }
            if let deviceId = DeviceManager.shared.deviceId {
                publicParams["devid"] = deviceId
            }
```

```
        if let vc = User.shared.verifyCode, !vc.isEmpty {
            publicParams["verifyCode"] = vc
        }
        switch self {
        case .registerDevice(let query, let body):
            query.forEach({publicParams[$0] = $1})
            return .requestCompositeParameters(bodyParameters: body, bodyEncoding:
URLEncoding.httpBody, urlParameters: publicParams)
        case .updateDevice(let query, let body):
            query.forEach({publicParams[$0] = $1})
            publicParams["t"] = String(format: "%.0f", Date().timeIntervalSince1970)
            let signContent = CryptoUtils.getPHPSignContent(query: publicParams, body:
body)
            if let sig = DeviceManager.shared.rawSign(content: signContent) {
                publicParams["c_sign"] = sig
            }
            return .requestCompositeParameters(bodyParameters: body, bodyEncoding:
URLEncoding.httpBody, urlParameters: publicParams)
        case .renew:
            return .requestParameters(parameters: publicParams, encoding:
URLEncoding.default)
        case .ud(let key, let cd):
            publicParams["key"] = key
            if let cd = cd {
                publicParams["cd"] = cd
            }
            return .requestParameters(parameters: publicParams, encoding:
URLEncoding.default)
        case .authTaoke(let authInfo):
            var bodyParam = authInfo.getInfoDict()
            let sign = CryptoUtils.PHPSign(params: bodyParam, key:
CryptoKeys.sign.rawValue)
            bodyParam["sn"] = sign
            var content = ""
            if let bodyParamData = try? JSONEncoder().encode(bodyParam), let
bodyParamString = String(data: bodyParamData, encoding: .utf8) {
                content = CryptoUtils.desEncrypt(key: CryptoKeys.content.rawValue,
value:bodyParamString) ?? ""
            }
            return .requestCompositeParameters(bodyParameters: ["content":content],
bodyEncoding: URLEncoding.httpBody, urlParameters: publicParams)
        case .v1Init:
            return .requestParameters(parameters: publicParams, encoding:
URLEncoding.default)
        case .appConfig:
            return .requestParameters(parameters: publicParams, encoding:
URLEncoding.default)
        case .v1Dispatch(let type, let content, let pasteInfo):
            publicParams["type"] = type.rawValue
            var bodyParam: [String: Any] = ["content":content]
            if let pasteInfo = pasteInfo {
```

```
                    bodyParam["pasteInfo"] = pasteInfo
                }
                return .requestCompositeParameters(bodyParameters: bodyParam, bodyEncoding:
JSONEncoding.default, urlParameters: publicParams)
            case .v1Clipboard(let content):
                var bodyParam = [String:Any]()
                bodyParam["content"] = content
                return .requestCompositeParameters(bodyParameters: bodyParam, bodyEncoding:
JSONEncoding.default, urlParameters: publicParams)
            case .appleAd(let body, let type):
                var query = [String:String]()
                query["t"] = String(format: "%.0f", Date().timeIntervalSince1970)
                query["type"] = String(type.rawValue)
                let sign = CryptoUtils.PHPSign(params: query, key: CryptoKeys.sign.rawValue)
//Crypto.sign(params: query, key: .sign)
                query["sn"] = sign
                query.forEach({publicParams[$0] = $1})
                return .requestCompositeParameters(bodyParameters: body, bodyEncoding:
URLEncoding.httpBody, urlParameters: publicParams)
            case .paste(let content):
                var bodyParam = [String:Any]()
                bodyParam["content"] = content
                return .requestCompositeParameters(bodyParameters: bodyParam, bodyEncoding:
JSONEncoding.default, urlParameters: publicParams)
            case .slog(let rt, let querys):
                publicParams["rt"] = rt
                if let querys = querys {
                    querys.forEach({publicParams[$0]=$1})
                }
                return .requestParameters(parameters: publicParams, encoding:
URLEncoding.default)
            case .notificationIncrement:
                return .requestCompositeParameters(bodyParameters: [String:Any](),
bodyEncoding: JSONEncoding.default, urlParameters: publicParams)
            case .start:
                return .requestParameters(parameters: publicParams, encoding:
URLEncoding.default)
            }
        }
        var headers: [String : String]? {
            return ["Accept":"application/json","User-Agent": UserAgent.shared.native]
        }
}
struct NetClient {
    static func defaultAlamofireSession() -> Session {
        let configuration = URLSessionConfiguration.default
        #if DEBUG || ADHOC
        let session = Session(configuration:configuration, startRequestsImmediately: false,
serverTrustManager: ServerTrustManager(evaluators: [
            phpHostStr:DisabledTrustEvaluator(),
            "gw.ruyisheng.com":DisabledTrustEvaluator(),
```

```
            "api.ruyisheng.com":DisabledTrustEvaluator(),
        ]))
        #else
        let session = Session(configuration: configuration, startRequestsImmediately: false)
        #endif
        return session
    }
    // MARK: - 设置请求超时时间
    #if DEBUG || ADHOC
    static let provider = MoyaProvider<NetAPI>(requestClosure: { (endpoint: Endpoint, done: @escaping MoyaProvider<NetAPI>.RequestResultClosure) in
        guard var request = try? endpoint.urlRequest() else {
            done(.failure(MoyaError.requestMapping(endpoint.url)))
            return
        }
        request.timeoutInterval = 60*2      //设置请求超时时间
        done(.success(request))
    }, session: defaultAlamofireSession(), plugins: [NetworkLoggerPlugin(configuration: .init(logOptions: .verbose))])
    #else
    static let provider = MoyaProvider<NetAPI>(requestClosure: { (endpoint: Endpoint, done: @escaping MoyaProvider<NetAPI>.RequestResultClosure) in
        guard var request = try? endpoint.urlRequest() else {
            done(.failure(MoyaError.requestMapping(endpoint.url)))
            return
        }
        request.timeoutInterval = 60*2      //设置请求超时时间
        done(.success(request))
    }, session: defaultAlamofireSession())
    #endif
    @discardableResult
    static func request<T:Decodable>(target: NetAPI,success successCallback: ((T, Response) -> Void)?, failure failureCallback: ((Error) -> Void)? = nil, completion completionCallback: (() -> Void)? = nil
    ) -> Cancellable {
        return provider.request(target) { result in
            DispatchQueue.main.async {
                switch result {
                case .success(let response):
                    do {
                        if let jsonDictionary = try response.mapJSON() as? [AnyHashable:Any], let status = jsonDictionary["status"] as? Int, status == 0 {
                            // 处理 errorCode 弹登陆
                            failureCallback?(NetworkError.invalidStatus(jsonDictionary))
                            return
                        }
                    } catch {
                        failureCallback?(error)
                        return
                    }
                    do {
```

```
                                let decoder = JSONDecoder()
                                decoder.dateDecodingStrategy = .iso8601
                                let data = try response.map(T.self, using: decoder)
                                successCallback?(data, response)
                            } catch {
                                failureCallback?(error)
                            }
                        case .failure(let error):
                            failureCallback?(error)
                        }
                        completionCallback?()
                    }
                }
            }
            @discardableResult
            static func requestJson(target: NetAPI,success successCallback: (([AnyHashable:Any]?, Response) -> Void)?, failure failureCallback: ((Error) -> Void)? = nil, completion completionCallback: (() -> Void)? = nil
            ) -> Cancellable {
                return provider.request(target) { result in
                    DispatchQueue.main.async {
                        switch result {
                        case .success(let response):
                            do {
                                if let jsonDictionary = try response.mapJSON() as? [AnyHashable:Any], let status = jsonDictionary["status"] as? Int, status == 0 {
                                    // 处理 errorCode 弹登陆
                                    failureCallback?(NetworkError.invalidStatus(jsonDictionary))
                                    return
                                }
                            } catch {
                                failureCallback?(error)
                                return
                            }
                            do {
                                let data = try response.mapJSON() as? [AnyHashable:Any]
                                successCallback?(data, response)
                            } catch {
                                failureCallback?(error)
                            }
                        case .failure(let error):
                            failureCallback?(error)
                        }
                        completionCallback?()
                    }
                }
            }
        }
        enum NetworkError: Error {
            case invalidStatus([AnyHashable:Any])
            case limitUse(Int,String?) // 限制使用
```

```
}
import Foundation
import Cache
import SDWebImage
import os
class CacheManager {
    static let shared = CacheManager()
    let storage = try! Storage<String,Data>(diskConfig: DiskConfig(name: "ruyisheng", directory: try! FileManager.default.url(for: .documentDirectory, in: .userDomainMask, appropriateFor: nil, create: true).appendingPathComponent("ruyisheng")), memoryConfig: MemoryConfig(), transformer: TransformerFactory.forData())
    public func save<T:Codable>(data: T, forKey key: String) {
        let dataStorage = storage.transformCodable(ofType: T.self)
        do {
            try dataStorage.setObject(data, forKey: key)
        } catch {
            os_log(.error,"CacheManager error:%@",error.localizedDescription)
        }
    }
    public func get<T:Codable>(forKey key: String) -> T? {
        let dataStorage = storage.transformCodable(ofType: T.self)
        var result:T?
        do {
            result = try dataStorage.object(forKey: key)
        } catch {
            os_log(.error,"CacheManager error:%@",error.localizedDescription)
        }
        return result
    }
    public func remove(forKey key: String) {
        do {
            try storage.removeObject(forKey: key)
        } catch {
            os_log(.error,"CacheManager error:%@",error.localizedDescription)
        }
    }
    public func saveData(data: Data, forKey key: String) {
        do {
            try storage.setObject(data, forKey: key)
        } catch {
            os_log(.error,"CacheManager error:%@",error.localizedDescription)
        }
    }
    public func getData(forKey key: String) -> Data? {
        var result: Data?
        do {
            result = try storage.object(forKey: key)
        } catch {
            os_log(.error,"CacheManager error:%@",error.localizedDescription)
        }
        return result
```

```
    }
}
protocol LocalCacheProtocol: Codable {
    @discardableResult
    static func loadFromLocal() -> Self?
    @discardableResult
    static func loadFromCache() -> Self?
    @discardableResult
    static func loadFromBundle() -> Self?
    /// - Returns: key
    static func localCacheKey() -> String
    func save()
}
extension LocalCacheProtocol {
    @discardableResult
    static func loadFromLocal() -> Self? {
        if let cache = loadFromCache() {
            return cache
        }
        if let bundleCache = loadFromBundle() {
            return bundleCache
        }
        return nil
    }
    @discardableResult
    static func loadFromCache() -> Self? {
        let key = localCacheKey()
        return CacheManager.shared.get(forKey: key)
    }
    func save() {
        let key = Self.localCacheKey()
        CacheManager.shared.save(data: self, forKey: key)
    }
    static func remove() {
        let key = localCacheKey()
        CacheManager.shared.remove(forKey: key)
    }
    @discardableResult
    static func loadFromBundle() -> Self? {
        let key = localCacheKey()
        guard let path = Bundle.main.path(forResource: key, ofType: "json") else {return nil}
        guard let data = try? Data(contentsOf:URL(fileURLWithPath: path)) else { return nil }
        do {
            let resp = try JSONDecoder().decode(Self.self, from: data)
            return resp
        } catch {
            os_log(.error,"CacheManager error:%@",error.localizedDescription)
        }
        return nil
    }
    static func localCacheKey() -> String {
```

```
            return String(describing: Self.self)
        }
}
import UIKit
import UserNotifications
import os
class PushManager: NSObject {
    static let shared = PushManager()
    func requestAuthorization() {
        let center = UNUserNotificationCenter.current()
        center.delegate = self
        center.requestAuthorization(options: [.alert, .badge, .sound, .carPlay]) { (granted, error)
in
            #if DEBUG || ADHOC
            os_log(.default, log:.normal,"Push Permission granted: %{public}@",(granted ?
"true" : "false"))
            #endif
        }
    }
    func startLocal(title: String?, subTitle: String?, body: String?, url: String?, timeInterval:
TimeInterval, identifier: String = ProcessInfo.processInfo.globallyUniqueString) {
        // Configure the notificaiton's payload.
        let content = UNMutableNotificationContent()
        if let title = title {
            content.title = title
        }
        if let subTitle = subTitle {
            content.subtitle = subTitle
        }
        if let body = body {
            content.body = body
        }
        if let url = url {
            content.userInfo["url"] = url
        }
        content.sound = UNNotificationSound.default
        // Create the request.
        let trigger = UNTimeIntervalNotificationTrigger(timeInterval: timeInterval, repeats:
false)
        let request = UNNotificationRequest(identifier: identifier, content: content, trigger:
trigger)
        UNUserNotificationCenter.current().add(request) { (error) in
            #if DEBUG || ADHOC
            if let error = error {
                os_log(.default,
log:.normal,"\n==========\nFailed to add request to notification center.\n notification title:%{public}@\n notification subtitle:%{public}@\n notification body:%{public}@\n notification url:%{public}@\n notification error:%@\n==========\n",
title ?? "", subTitle ?? "", body ?? "", url ?? "" ,error.localizedDescription)
            }
```

```
                os_log(.default,
log:.normal,"\n==========\nSuccess add local request to notification center.\n notification title:%{public}@\n notification subtitle:%{public}@\n notification body:%{public}@\n notification url:%{public}@\n==========", title ?? "", subTitle ?? "", body ?? "", url ?? "")
                #endif
            }
        }
}
extension PushManager: UNUserNotificationCenterDelegate {
    func userNotificationCenter(_ center: UNUserNotificationCenter, willPresent notification: UNNotification, withCompletionHandler completionHandler: @escaping (UNNotificationPresentationOptions) -> Void) {
        if #available(iOS 14.0, *) {
            completionHandler([.sound,.badge,.banner,.list])
        } else {
            completionHandler([.sound,.badge,.alert])
        }
        handle(notification: notification, isFromForground: true)
    }
    func userNotificationCenter(_ center: UNUserNotificationCenter, didReceive response: UNNotificationResponse, withCompletionHandler completionHandler: @escaping () -> Void) {
        completionHandler()
        handle(notification: response.notification, isFromForground: false)
    }
    func handle(notification: UNNotification, isFromForground: Bool) {
        guard !isFromForground else {
            return // 前台推送不响应 url，否则前台一推送直接自动跳转了
        }
        // 处理推送的 userInfo
        if let urlStr = notification.request.content.userInfo["url"] as? String, let url = URL(string: urlStr), let scheme = SchemeHandler(url) {
            guard PageManager.getRootNav() != nil else {
                let appDelegate = UIApplication.shared.delegate as? AppDelegate
                appDelegate?.pendingSchemes.append(scheme)
                return
            }
            scheme.execute(SchemeContext(controller: PageManager.getTop()))
        }
        /*if notification.request.trigger is UNPushNotificationTrigger { // 处理远程推送
        } else { // 本地推送
        }*/
    }
}
import Foundation
import Alamofire
import os
struct Track {
    static func event(_ event: String, subEvent: String? = nil, attributes: [String: String]? = nil) {
        let data = TrackData(eventId: event, eventSubId: subEvent, attributes: attributes)
```

```
            TrackManager.shared.saveTrack(data)
        }
        static func btnClick(btnName:String, subEvent: String? = nil, attributes: [String: String]? =
nil) {
            var info = attributes ?? [String:String]()
            info["btn_name"] = btnName
            event("btn_click", subEvent: subEvent, attributes: info)
        }
}
class TrackManager {
        static let shared = TrackManager()
        var fileHandler = TrackFileHandler()
        var cacheDispath: DispatchUploadData?
        var tempCache = [TrackData]()
        var uploadingList: Set = Set<URL>()
        var uuid: String = ""
        private var appOpen = false
        var timer: Timer?
        private lazy var saveTrackQueue: OperationQueue = {
            let queue = OperationQueue()
            queue.maxConcurrentOperationCount = 1
            return queue
        }()
        init() {
            cacheDispath = CacheManager.shared.get(forKey: "track_dispatch_resp_data")
        }
        func saveTrack(_ track: TrackData?) {
            guard let track = track else { return }
            if appOpen {
                saveTrackQueue.addOperation {[weak self] in
                    self?.fileHandler.write(track: track)
                }
            } else {
                tempCache.append(track)
            }
        }
        func saveTracks(_ tracks: [TrackData]?) {
            guard let tracks = tracks else { return }
            if appOpen {
                saveTrackQueue.addOperation {[weak self] in
                    self?.fileHandler.write(array: tracks)
                }
            } else {
                tempCache.append(contentsOf: tracks)
            }
        }
        @objc func openApp() {
            let uuidStr = UUID().uuidString
            uuid = uuidStr
            cleanContext {[weak self] in
                self?.startTimer() //  启动后 10 秒后上传
```

```
                self?.appOpen = true
                self?.uploadingList.removeAll()
                self?.fileHandler.openHandler()
                Track.event("app_open", attributes:
                                [
                                    "device_type":UIDevice.current.model,
                                    "model":DeviceManager.shared.deviceModel ?? "",
                                    "idfa":DeviceManager.shared.idfa ?? "",
                                    "mac":DeviceManager.shared.macAddress ?? "",
                                    "uuid":uuidStr,
                                    "type":DeviceManager.shared.startByInit ? "0" : "1"
                                ])
                self?.saveTracks(self?.tempCache) // appOpen==false 下暂存的 track 都写入了，
然后清空 tempCache
                self?.tempCache.removeAll()
            }
        }
        @objc func closeApp() {
            Track.event("app_close", attributes: ["uuid":uuid])
            appOpen = false
            cleanContext { [weak self] in
                self?.startUpload(ignoreCurrent: false, completion: nil)
            }
            stopTimer() // 退后台取消延时上传
        }
        func cleanContext(completion:(() -> Void)?) {
            guard let _ = fileHandler.handler else {
                moveCurrentFile(completion: completion)
                return
            }
            saveTrackQueue.addOperation {[weak self] in
                self?.fileHandler.closeHandler()
                OperationQueue.main.addOperation {
                    self?.moveCurrentFile(completion: completion)
                }
            }
        }
        func moveCurrentFile(completion:(() -> Void)?) {
            if FileManager.default.fileExists(atPath: fileHandler.filePath.filePathString) {
                let lastPath = fileHandler.filePath.lastPathComponent
                let newLastPath = lastPath.replacingOccurrences(of: ".data", with: String(format:
"-%.0f.data", Date().timeIntervalSince1970))
                var newFilePath = fileHandler.filePath
                newFilePath.deleteLastPathComponent()
                if #available(iOS 16.0, *) {
                    newFilePath.append(path: newLastPath, directoryHint: .notDirectory)
                } else {
                    newFilePath.appendPathComponent(newLastPath, isDirectory: false)
                }
                do {
                    try FileManager.default.moveItem(at: fileHandler.filePath, to: newFilePath)
```

```
                } catch {
                    print("moveItem error:\(error)")
                }
            }
            completion?()
        }
        func startUpload(ignoreCurrent: Bool, completion: ((Bool) -> Void)?) {
            guard let url = fileToUpload(ignoreCurrent: ignoreCurrent) else {
                completion?(false)
                return
            }
            requestUploadToken {[weak self] uploadData in
                if let open = uploadData.open, open == 1, let apiPath = uploadData.url, !apiPath.isEmpty {
                    self?.uploadTrack(path: url, url: apiPath) { success in
                        // 日志上传完毕
                        os_log(.default, log:.normal,"close App, upload track finish")
                    }
                } else { // 过期日志删除放在 不允许上传的逻辑里
                    self?.checkLocalFileStatus()
                    if let interval = uploadData.interval { // 延迟上传
                        self?.startTimer(interval: interval)
                    }
                }
            }
        }
        func requestUploadToken(completion:((DispatchUploadData) -> Void)?) {
            if let cache = cacheDispath, let cacheTime = cache.cacheTime, Date(timeIntervalSince1970: cacheTime) >= Date() {
                completion?(cache) // 有缓存，且时间是有效的，就用缓存的
                return
            }
            self.cacheDispath = nil // 清除缓存数据，再请求新数据
            AF.request("https://api.ruyisheng.com/app/v1/dispatch.htm",
                       method: .get,
                       parameters: [
                        "src":sourceStr,
                        "v":Utils.marketVersion(),
                        "type":"2",
                        "uid":User.shared.userId ?? "",
                        "devid":DeviceManager.shared.deviceId ?? ""],
                       headers: [.accept("application/json"),.userAgent(UserAgent.shared.native)]).responseDecodable(of: DispatchUploadResponse.self) { dataResponse in
                switch dataResponse.result {
                case .success(let resp):
                    self.cacheDispath = resp.data
                    if let data = resp.data { // 缓存
                        CacheManager.shared.save(data: data, forKey: "track_dispatch_resp_data")
                    }
```

```
                    if let d = resp.data {
                        completion?(d)
                    }
                case .failure(let error):
                    print("/app/v1/dispatch failed: \(error.localizedDescription)")
                    break
                }
            }
        }
        func uploadTrack(path: URL, url: String, completion: ((Bool) -> Void)?) {
            guard !uploadingList.contains(path), let trackList = trackLists(), !trackList.isEmpty else
{
                completion?(false)
                return
            }
            let currentFile = fileHandler.fileName
            if trackList.count > 2 {
                handlerMutilFile(path: path, list: trackList, curName: currentFile)
            }
            uploadingList.insert(path)
            guard let component = URLComponents(string: url) else {
                completion?(false)
                return
            }
            var requestURL = component
            requestURL.queryItems?.append(URLQueryItem(name: "t", value: String(format:
"%.0f", Date().timeIntervalSince1970)))
            requestURL.percentEncodedQueryItems = queryItems
            var bodyData: Data?
            do {
                bodyData = try Data(contentsOf: path)
            } catch {
                print("Data(contentsOf: path) failed, error:\(error)")
            }
            guard let bodyData = AppSign.cryptBody(body: bodyData) else { // 给 body 加密
                completion?(false)
                return
            }
            AF.upload(bodyData, to: requestURL, method: .post, headers:
[.contentType("application/octet-stream"),.defaultAcceptEncoding,.defaultAcceptLanguage,.userA
gent(UserAgent.shared.native),.accept("application/json")]).responseDecodable(of:
TrackUploadResponse.self) {[weak self] response in
                switch response.result {
                case .success(let resp):
                    self?.uploadingList.remove(path)
                    guard let status = resp.status, status == "1" else {
                        completion?(false)
                        return
                    }
                    do {
                        try FileManager.default.removeItem(at: path)
```

```
                } catch {
                    print("FileManager.default.removeItem(at: path), error: \(error)")
                }
                if let nextFile = self?.fileToUpload(ignoreCurrent: true) {
                    self?.uploadTrack(path: nextFile, url: url, completion: completion)
                } else {
                    completion?(true)
                }
            case .failure(let error):
                self?.uploadingList.remove(path)
                print("upload track to \(url) failed, error:\(error.localizedDescription)")
            }
        }
    }
    func handlerMutilFile(path: URL, list: [URL], curName: String) {
        guard let handler = try? FileHandle(forUpdating: path) else { return }
        var seekEndOffset: UInt64 = 0
        if #available(iOS 13.4, *) {
            do {
                seekEndOffset = try handler.seekToEnd()
            } catch {
                print("seekToEnd error:\(error)")
            }
        } else {
            seekEndOffset = handler.seekToEndOfFile()
        }
        for filePath in list {
            if filePath.lastPathComponent == curName || filePath == path {
                continue
            } else if seekEndOffset > 1024 * 1024 {
                break
            } else {
                if let reader = try? FileHandle(forUpdating: filePath) {
                    // read
                    var data: Data?
                    if #available(iOS 13.4, *) {
                        do {
                            data = try reader.readToEnd()
                        } catch {
                            print("readToEnd error:\(error)")
                        }
                    } else {
                        data = reader.readDataToEndOfFile()
                    }
                    // write
                    if let data = data {
                        if #available(iOS 13.4, *) {
                            try? handler.write(contentsOf: data)
                        } else {
                            handler.write(data)
                        }
```

```
                        if #available(iOS 13.4, *) {
                            do {
                                seekEndOffset = try handler.offset()
                            } catch    {
                                print("offset error:\(error)")
                            }
                        } else {
                            seekEndOffset = handler.offsetInFile
                        }
                    }
                    // close
                    if #available(iOS 13.0, *) {
                        try? reader.close()
                    } else {
                        reader.closeFile()
                    }
                    do {
                        try FileManager.default.removeItem(at: filePath)
                    } catch {
                        print("FileManager.default.removeItem(atPath:                readPath)
error:\(error)")
                    }
                }
            }
        }
    }
    func trackLists() -> [URL]? {
        do {
            let fileContents = try FileManager.default.contentsOfDirectory(at:
fileHandler.fileDirectory, includingPropertiesForKeys: [URLResourceKey.creationDateKey],
options: .skipsSubdirectoryDescendants)
            let result = fileContents.filter({$0.pathExtension == "data"}).sorted { aUrl, bUrl in
                let lastA = aUrl.lastPathComponent
                let lastB = bUrl.lastPathComponent
                return lastA < lastB
            }
            return result
        } catch {
            print("contentsOfDirectory error:\(error)")
        }
        return nil
    }
    func fileToUpload(ignoreCurrent: Bool) -> URL? {
        guard let trackList = trackLists(), !trackList.isEmpty else {return nil}
        if !ignoreCurrent {
            return trackList.first
        }
        return filePathNeedUpload(filePaths: trackList)
    }
    func filePathNeedUpload(filePaths: [URL]?) -> URL? {
        guard let filePaths = filePaths, !filePaths.isEmpty else { return nil }
```

```
            guard let target = filePaths.filter({!$0.lastPathComponent.contains(fileHandler.fileName)}).first else {return nil}
            return target
        }
        func checkLocalFileStatus() {
            guard let trackList = trackLists(), !trackList.isEmpty else {return}
            let cacheDay: TimeInterval = 3 * 24 * 3600 // 3 天
            let latestTime = Date(timeInterval: -(cacheDay), since: Date()) // 3 天前的时间
            let lastFileName = fileHandler.fileName(date: latestTime)
            for filePath in trackList {
                let lastPath = filePath.lastPathComponent
                if lastPath < lastFileName {
                    do {
                        try FileManager.default.removeItem(at: filePath)
                    } catch {
                        print("FileManager.default.removeItem(at: filePath), error:\(error)")
                    }
                }
            }
        }
        // MARK: - Timer
        func startTimer(interval: TimeInterval = 10) {
            stopTimer()
            timer = Timer.scheduledTimer(withTimeInterval: interval, repeats: false, block: {[weak self] t in
                t.invalidate()
                self?.startUpload(ignoreCurrent: true, completion: nil)
                self?.timer = nil
            })
            if let t = timer {
                RunLoop.current.add(t, forMode: .common)
            }
        }
        func stopTimer() {
            if let t = timer {
                t.invalidate()
                timer = nil
            }
        }
}
import Foundation
struct TrackData: CustomStringConvertible {
        let eventId: String
        let eventSubId: String?
        let eventData: String?
        let timestamp: TimeInterval = Date().timeIntervalSince1970
        var localTime: TimeInterval {
            round(timestamp)
        }
        init(eventId: String, eventSubId: String? = nil, attributes: [String: String]? = nil) {
            self.eventId = eventId
```

```
            self.eventSubId = eventSubId
            if let infos = attributes, !infos.isEmpty {
                let infoString = infos.reduce("") { partialResult, element in
                    var partialResult = partialResult
                    if !partialResult.isEmpty {
                        partialResult += "&"
                    }
                    partialResult += ((element.key.urlEncode() ?? "") + "=" +
(element.value.urlEncode() ?? ""))
                    return partialResult
                }
                self.eventData = infoString
            } else {
                self.eventData = nil
            }
        }
        var description: String {
            let uid = User.shared.userId ?? ""
            let devId = DeviceManager.shared.deviceId ?? ""
            let appVersion = Utils.marketVersion()
            return String(format:"%@,%@,%@,%.0lf,%.0lf,%@,%@,%@,%@,%@\n" , eventId,
eventSubId ?? "", eventData ?? "", timestamp, localTime, uid, devId, sourceStr, appVersion,
channel)
        }
}
struct DispatchUploadResponse: Codable {
        let data: DispatchUploadData?
}
struct DispatchUploadData: Codable {
        let open: Int?
        let interval: TimeInterval?
        let url: String?
        let cacheTime: TimeInterval?
}
struct TrackUploadResponse: Codable {
        let info: String?
        let status: String?
}
class TrackFileHandler {
        let fileName: String
        let fileDirectory: URL
        let filePath: URL
        var handler: FileHandle?
        let df = DateFormatter()
        init() {
            df.dateFormat = "yyyy-MM-dd"
            fileName = df.string(from: Date())+".data"
            let document = FileManager.default.urls(for: .documentDirectory,
in: .userDomainMask).first!
            if #available(iOS 16.0, *) {
                fileDirectory = document.appending(path: "track", directoryHint: .isDirectory)
```

```
            } else {
                fileDirectory = document.appendingPathComponent("track", isDirectory: true)
            }
            if #available(iOS 16.0, *) {
                filePath = fileDirectory.appending(component: fileName,
directoryHint: .notDirectory)
            } else {
                filePath = fileDirectory.appendingPathComponent(fileName, isDirectory: false)
            }
        }
        deinit {
            closeHandler()
        }
        func write(track: TrackData) {
            write(data: track.description.data(using: .utf8))
        }
        func write(array: [TrackData]) {
            array.forEach({self.write(track: $0)})
        }
        func fileName(date: Date) -> String {
            return df.string(from: date)+".data"
        }
        func openHandler() {
            if let _ = handler { // cleanup
                closeHandler()
            }
            if !FileManager.default.fileExists(atPath: fileDirectory.path) {
                do {
                    try FileManager.default.createDirectory(at: fileDirectory,
withIntermediateDirectories: true)
                } catch {
                    print("createDirectory failed error:\(error) ")
                }
            }
            let path = filePath.filePathString
            if !FileManager.default.fileExists(atPath: path) {
                let result = FileManager.default.createFile(atPath: path, contents: nil)
                if !result {
                    print("FileHandle createFile failed")
                }
            }
            do {
                handler = try FileHandle(forUpdating: filePath)
            } catch {
                print("FileHandle(forUpdating: filePath) error:\(error)")
            }
            handlerSeekToEnd()
        }
        func closeHandler() {
            guard let handler = handler else { return }
            if #available(iOS 13.0, *) {
```

```
                do {
                    try handler.close()
                } catch {
                    print("handler?.close(), error:\(error)")
                }
            } else {
                handler.closeFile()
            }
            self.handler = nil
        }
        func write(data: Data?) {
            guard let data = data, let handler = handler else { return }
            if #available(iOS 13.4, *) {
                do {
                    try handler.write(contentsOf: data)
                } catch {
                    print("write data error: \(error)")
                }
            } else {
                handler.write(data)
            }
        }
        @discardableResult
        func handlerSeekToEnd() -> UInt64 {
            guard let handler = handler else { return 0 }
            if #available(iOS 13.4, *) {
                do {
                    return try handler.seekToEnd()
                } catch {
                    print("seekToEnd error: \(error)")
                    return 0
                }
            } else {
                return handler.seekToEndOfFile()
            }
        }
}
extension URL {
        var filePathString: String {
            var path: String
            if #available(iOS 16.0, *) {
                path = self.path()
            } else {
                path = self.path
            }
            return path
        }
}
```